

PROJECT DOCUMENTATION

PROJECT TITLE

MULTI-LINE INSURANCE POLICY AND CLAIMS MANAGEMENT SYSTEM

Team ID : SWTID-2026-9303

Team Size : 5

Team Leader : DEBORA SHALINI T

Team member : SHEELALAKSHMI G

Team member : LAVANYA K

Team member : LATHIKA M

Team member : ROHINI M

SUBMITTED BY,

NAME : DEBORA SHALINI T

REGISTER NO : 920123104006

NM ID : 4FCDACC4206B5259C9E4D187947B2698

User Story

As an insurance customer,
I want to manage my insurance policies, submit and track claims, and receive updates
on my policy and claim status,
so that I can easily manage my insurance needs and get faster and more transparent
claim processing.

Key User Stories

Policy Management

As an insurance customer, I want to view my active policies and their details so that I
can keep track of my insurance coverage.

Claim Submission

As an insurance customer, I want to submit a claim with the required details and
documents so that my claim can be processed.

Claim Tracking

As an insurance customer, I want to track the status of my claim so that I know whether
it is submitted, under review, approved, or rejected.

Policy Verification

As an insurance agent, I want to verify policy details and customer information so that
claims are processed against valid policies.

Claim Approval

As a claims officer, I want to review and approve or reject claims based on policy
coverage and claim information so that claims are handled correctly.

Automated Updates

As a customer, I want to receive notifications when my policy or claim status changes so
that I stay informed.

Dashboard and Reporting

As an administrator, I want to view policy and claim information through dashboards
and reports so that I can monitor the overall insurance operations.

TABLE OF CONTENTS

1.INTRODUCTION

2.IDEATION PHASE

3.REQUIREMENT ANALYSIS PHASE

4.PROJECT DESIGN PHASE

5.PROJECT PLANNING AND SCHEDULING

6.PROJECT DEVELOPMENT PHASE

7.FUNCTIONAL AND PERFORMANCE TESTING

8.ADVANTAGES AND DISADVANTAGES

9.CONCLUSION

10.FUTURE SCOPE

11.APENDIX

1. INTRODUCTION

1.1 Project Overview

The Multi-Line Insurance Policy and Claims Management System is a Salesforce-based insurance management solution designed to manage multiple types of insurance policies and their associated claims.

Insurance organizations deal with large volumes of customer information, policy records, claims, claim assignments, and processing activities. Managing these processes manually can result in delays, duplicate data, incorrect claim routing, and difficulty monitoring claim status.

The proposed system provides a centralized platform where insurance policies, policy holders, and claims can be managed efficiently.

The project is developed through four major milestones:

Milestone 1 – Core Data Model & Policy Configuration

This milestone focuses on creating the basic Salesforce data model and configuring the required policy-related information.

Major activities include:

- **Creating insurance-related objects and fields.**
- **Configuring policy information.**
- **Maintaining policy holder details.**
- **Establishing relationships between records.**
- **Creating the basic structure required for claim processing.**

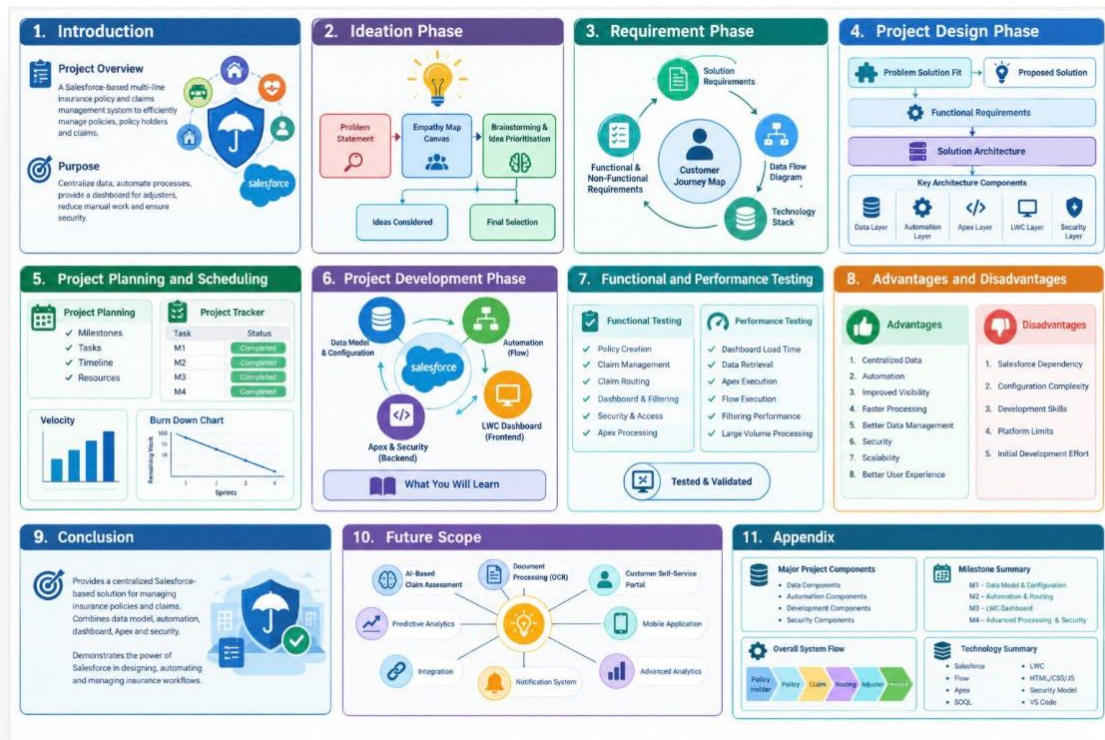
Milestone 2 – Complex Policy Insurance & Claim Routing Automation

This milestone introduces automation into the insurance and claims process.

Major activities include:

- **Automating claim-related processes.**
- **Applying business rules.**
- **Routing claims based on defined conditions.**
- **Assigning claims to appropriate queues/users.**
- **Updating relevant claim information automatically.**

➤ **Reducing manual processing.**



Milestone 3 – Claims Adjuster LWC Dashboard Development

This milestone focuses on developing a user-friendly dashboard for claims adjusters using Lightning Web Components (LWC).

The dashboard allows users to:

- View claims.
- Monitor claim information.
- Filter claims.
- Access relevant claim details.
- Work with claim data through a centralized interface.

Milestone 4 – Advanced Claim Processing, Security & Code Setup

The final milestone focuses on advanced development and system protection.

Activities include:

- Advanced claim processing.
- Apex development.

- LWC integration.
- Security configuration.
- Access control.
- Validation and testing.
- Improving system reliability and maintainability.

1.2 Purpose

The main purpose of the project is to create an efficient and centralized insurance policy and claims management system.

The system aims to:

- Manage multiple insurance policies.
- Maintain accurate policy holder information.
- Centralize claim information.
- Automate claim routing.
- Reduce manual work.
- Provide claims adjusters with an interactive dashboard.
- Improve claim processing efficiency.
- Protect sensitive insurance information.
- Provide a scalable Salesforce-based solution.

2. IDEATION PHASE

2.1 Problem Statement

Traditional insurance claim management can involve multiple manual activities such as collecting policy information, validating claims, assigning claims to employees, monitoring claim status, and updating records.



PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Insurance Operations Manager	standardize policy quoting and issuance	the process is manual, slow, and inconsistent	legacy systems and processes are fragmented	frustrated and concerned
PS-2	Insurance Agent	create and manage policies efficiently	different insurance lines require different information	Vehicle, Property, and Life policies have diverse data requirements	overwhelmed
PS-3	Insurance Agent	provide accurate policy quotes	quoting requires multiple steps and manual processing	there is no unified quoting platform	frustrated and uncertain
PS-4	Claims Manager	manage claims efficiently	claim review and routing are largely manual	claims processes are not centralized or automated	stressed
PS-5	Claims Adjuster	manage assigned claims effectively	I lack a focused view of my workload	claim information is not easily available in one place	frustrated and less productive
PS-6	Claims Manager	route claims to the appropriate team	claims need to be handled according to policy type and claim severity	routing processes are complex and require automation	concerned about delays
PS-7	Senior Adjuster	process high-value claims efficiently	approval requires multiple levels of review	high-value claims require structured approval and escalation	pressured
PS-8	Insurance Operations Manager	maintain accurate policy information	different products require different fields and validations	Vehicle, Property, and Life policies have varying data requirements	concerned about data quality
PS-9	Customer	get my policy and claim handled quickly	manual processing causes delays	policy and claims processes are fragmented	uncertain and dissatisfied
PS-10	Insurance Management	get a complete view of insurance operations	information is spread across different processes and systems	there is no unified platform for policy and claim management	concerned about operational efficiency

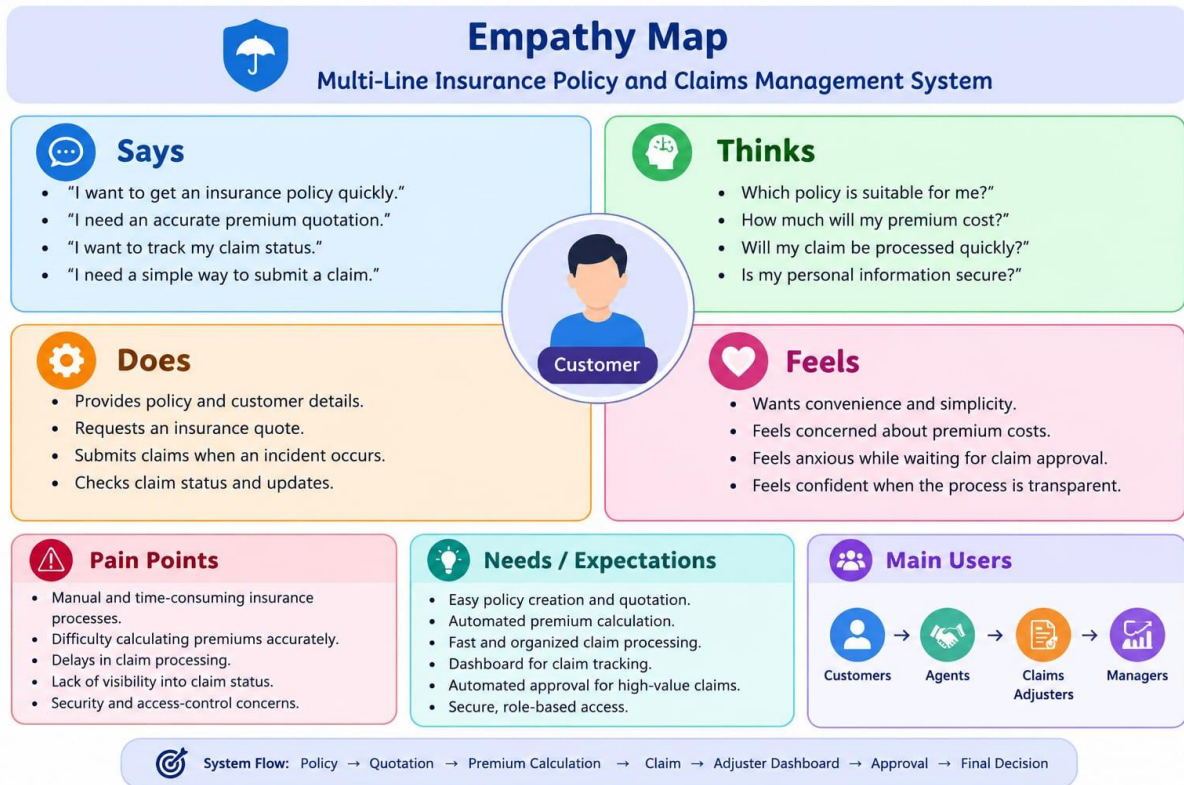
These processes may create problems such as:

- **Delayed claim processing.**
- **Manual errors.**
- **Difficulty locating policy information.**
- **Incorrect claim assignment.**
- **Lack of centralized information.**
- **Limited visibility into claim status.**
- **Repetitive administrative activities.**
- **Difficulty monitoring large numbers of claims.**

Therefore, there is a need for a centralized system that combines policy management, claim management, automation, dashboard visualization, and security.

2.2 Empathy Map Canvas

An empathy map helps understand the needs and challenges of the users who interact with the system.



User

Claims Adjuster

Insurance Administrator

Says

I need complete claim information before processing a claim.

Policy and customer information should be properly organized.

Thinks

I need to process claims quickly and avoid mistakes.

The system should reduce duplicate and incorrect data.

Does

Reviews claims, checks policy details, updates status and processes assigned claims.

Creates policies, manages records and monitors insurance information.

Feels

Frustrated when information is scattered or claims are incorrectly assigned.

Concerned about data accuracy and security.

Pain Points

Manual searching, incomplete information, incorrect routing, repetitive tasks.

Manual data entry and managing large amounts of information.

Needs

Centralized information, automation, filtering and dashboard visibility.

Structured data model, validation, automation and security.

2.3 Brainstorming and Idea Prioritisation

During brainstorming, different solutions were considered to address insurance policy and claim management challenges.

Ideas Considered

- **Manual Excel-based insurance management.**
- **Standalone insurance database.**
- **Web-based insurance management application.**
- **Salesforce-based insurance policy management.**
- **Salesforce-based claims management.**
- **Automated claim routing.**
- **Claims dashboard using LWC.**
- **Combined multi-line policy and claims management system.**

Final Selection

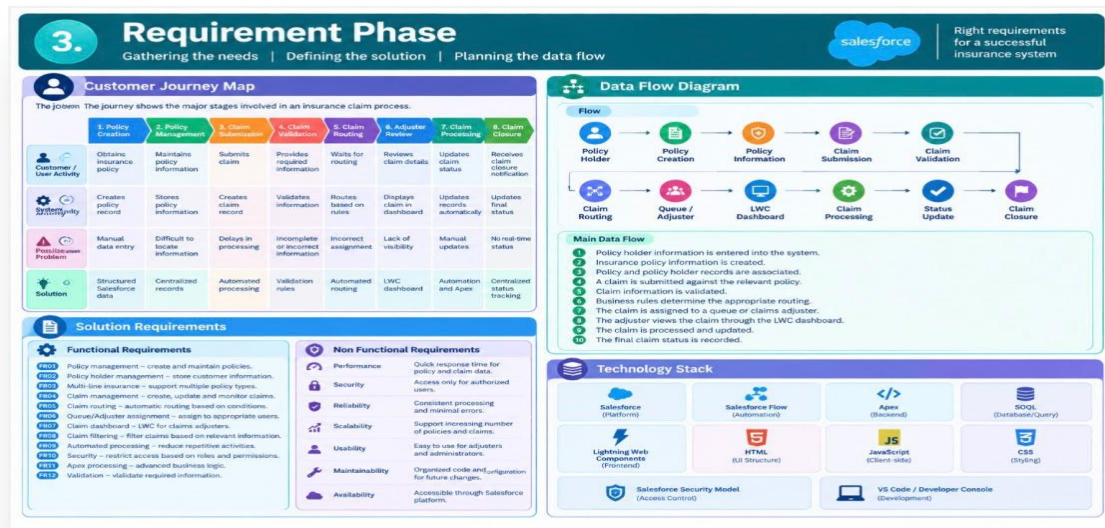
The final idea selected was:

- **Multi-Line Insurance Policy and Claims Management System**
- **The solution combines:**
- **Insurance policy management.**
- **Policy holder management.**
- **Claim management.**
- **Automated claim routing.**
- **Claims dashboard.**
- **Advanced claim processing.**

- Security.
- Apex and LWC development.

This approach provides a complete end-to-end insurance management solution within Salesforce.

3. REQUIREMENT PHASE



3.1 Customer Journey Map

The customer journey describes the major stages involved in an insurance claim process.

Stage

- Customer/User Activity
- System Activity
- Possible Problem

Solution

1. Policy Creation

- Customer obtains insurance policy
- Policy record is created
- Manual data entry
- Structured Salesforce data

2. Policy Management

- Customer maintains policy
- Policy information is stored
- Information may be difficult to locate

- **Centralized records**

3. Claim Submission

- **Customer submits claim**
- **Claim record is created**
- **Delays in processing**
- **Automated processing**

4. Claim Validation

- **Claim information is checked**
- **System validates relevant information**
- **Incorrect/incomplete information**
- **Validation rules**

5. Claim Routing

- **Claim requires processing**
- **System routes claim**
- **Incorrect assignment**
- **Automated routing**

6. Adjuster Review

- **Adjuster reviews claim**
- **Dashboard displays claim**
- **Difficulty tracking claims**
- **LWC dashboard**

7. Claim Processing

- **Adjuster updates claim**
- **System records changes**
- **Manual updates**
- **Automation**

8. Claim Closure

- **Claim is completed**
- **Status is updated**
- **Lack of visibility**
- **Centralized status tracking**

Stage	Journey Stage	Customer/User Activity	System Activity	Possible Problem	Solution
1	Policy Creation	Obtains insurance policy	Creates policy record	Manual data entry	Structured Salesforce data
2	Policy Management	Maintains policy information	Stores policy information	Difficulty locating information	Centralized records
3	Claim Submission	Submits claim	Creates claim record	Delays in processing	Automated processing
4	Claim Validation	Provides required information	Validates information	Incomplete/incorrect information	Validation rules
5	Claim Routing	Waits for claim assignment	Routes claim based on rules	Incorrect assignment	Automated routing
6	Adjuster Review	Reviews claim details	Displays claim in dashboard	Lack of visibility	LWC dashboard
7	Claim Processing	Updates claim status	Updates records automatically	Manual updates	Automation and Apex
8	Claim Closure	Receives closure notification	Updates final status	No real-time status	Centralized status tracking

3.2 Solution Requirements

- **Functional Requirements**
- **Non Functional Requirements**

Functional Requirements

ID	Functional Requirement	Description
FR01	Policy Management	The system shall allow authorized users to create, view, update, and manage insurance policies.
FR02	Policy Holder Management	The system shall store and manage policy holder/customer information.
FR03	Multi-Line Insurance	The system shall support multiple types of insurance policies.
FR04	Claim Management	The system shall allow users to create, view, update, and monitor insurance claims.
FR05	Claim Routing	The system shall automatically route claims based on predefined business conditions.
FR06	Queue/Adjuster Assignment	The system shall assign claims to the appropriate queue or claims adjuster.
FR07	Claims Dashboard	The system shall provide an LWC dashboard for claims adjusters to view claim information.
FR08	Claim Filtering	The system shall allow claims adjusters to filter and search claims based on relevant information.
FR09	Automated Processing	The system shall automate repetitive claim-related activities and field updates.
FR10	Security and Access Control	The system shall restrict access to insurance information based on user permissions and access levels.
FR11	Apex Processing	The system shall use Apex to perform advanced claim processing and business logic where required.
FR12	Data Validation	The system shall validate required information before policy and claim records are processed.

3.3 Non-Functional Requirements

ID	Category	Non-Functional Requirement
NFR01	Performance	The system should provide quick response times when users access policy and claim information.
NFR02	Security	Sensitive insurance and customer information should be accessible only to authorized users.
NFR03	Reliability	The system should process policy and claim records consistently with minimal errors.
NFR04	Scalability	The system should support an increasing number of policies, customers, and claims.
NFR05	Usability	The system should provide a simple and user-friendly interface for claims adjusters and administrators.
NFR06	Maintainability	The Salesforce configuration, Apex code, and LWC components should be organized for easy maintenance and future changes.
NFR07	Availability	The system should be available to authorized users through the Salesforce platform.
NFR08	Data Integrity	The system should maintain accurate, consistent, and reliable policy and claim information.
NFR09	Efficiency	Automation should reduce repetitive manual tasks and improve claim processing efficiency.
NFR10	Extensibility	The system should allow additional insurance products, features, and business processes to be added in the future.

3.4 Data Flow Diagram

Flow

The overall system flow can be represented as:

Policy Holder → Policy Creation → Policy Record → Claim Submission → Claim Validation → Claim Routing → Adjuster/Queue → Claim Processing → Claim Status Update → Claim Closure

Main Data Flow

- Policy holder information is entered into the system.
- Insurance policy information is created.
- Policy and policy holder records are associated.
- A claim is submitted against the relevant policy.
- Claim information is validated.
- Business rules determine the appropriate routing.
- The claim is assigned to a queue or claims adjuster.
- The adjuster views the claim through the LWC dashboard.
- The claim is processed and updated.
- The final claim status is recorded.

3.5 Technology Stack

Technology	Category	Purpose
Salesforce	Cloud Platform	Main platform for insurance policy and claims management.
Salesforce Objects	Data Storage	Store policy, policy holder and claim records.
Salesforce Flow	Automation	Automate business processes, routing and record updates.
Apex	Backend	Implement advanced server-side business logic and claim processing.
SOQL	Database Query	Retrieve Salesforce records.
Lightning Web Components	Frontend	Build the claims adjuster dashboard and interactive interface.
HTML	UI Structure	Define LWC user interface structure.
JavaScript	Client-side Logic	Implement dashboard interactions and filtering.
CSS	UI Styling	Style the dashboard.
Salesforce Security Model	Security	Control access to sensitive insurance information.
VS Code / Developer Console	Development	Develop, test and manage Salesforce code and configuration.

4. PROJECT DESIGN PHASE



4.1 Problem-Solution Fit

Problem

Insurance organizations need to manage large amounts of policy and claim information while ensuring that claims are processed efficiently and securely.

Proposed Solution

The project provides a centralized Salesforce system that connects:

Policy → Policy Holder → Claim → Routing → Adjuster → Processing → Status

The solution addresses the identified problems by:

- **Centralizing information.**
- **Automating claim routing.**
- **Reducing manual work.**
- **Providing dashboard visibility.**
- **Supporting advanced processing.**
- **Applying security controls.**

4.2 Proposed Solution

The proposed system consists of four major layers corresponding to the project milestones.

Layer 1 – Data Model

Provides:

- **Policy records.**
- **Policy holder information.**
- **Claim records.**
- **Relationships.**

Required fields.

Layer 2 – Automation

Provides:

- **Policy-based processing.**
- **Claim routing.**
- **Automated updates.**
- **Business rules.**

Layer 3 – User Interface

Provides:

- **Claims adjuster dashboard.**

- **Claim list.**
- **Filtering.**
- **Claim information display.**

Layer 4 – Advanced Processing and Security

Provides:

- **Apex logic.**
- **Advanced claim processing.**
- **Security.**
- **Access control.**
- **Testing.**

4.3 Functional Requirements of Proposed Solution

The proposed system must:

- **Create and manage insurance policies.**
- **Maintain policy holder information.**
- **Support multiple insurance lines.**
- **Create and manage claims.**
- **Validate claim information.**
- **Automatically route claims.**
- **Assign claims to appropriate users/queues.**
- **Display claims through an LWC dashboard.**
- **Allow client-side filtering.**
- **Process claims using automation and Apex.**
- **Secure insurance information.**
- **Track claim status throughout its lifecycle.**

4.4 Solution Architecture

The architecture follows a Salesforce-centered approach.

Architecture Component	Key Components	Purpose
Users	Admin, Claims Processor, Claims Adjuster	Interact with policy and claim processes.
Salesforce Data Layer	Policy, Policy Holder, Claim and related records	Stores structured insurance information.
Automation Layer	Salesforce Flow, routing rules, field updates	Automates business processes and claim routing.
Apex Layer	Apex classes and SOQL	Handles advanced business logic and server-side processing.
LWC Layer	Claims Adjuster Dashboard	Provides an interactive user interface and filtering.
Security Layer	Profiles, Permission Sets, Object/Field/Record access	Protects sensitive insurance data.
End-to-End Flow	Users → Data → Automation → Apex → LWC	Connects the major architecture components.

Key Architecture Components

1. Salesforce Data Layer

Stores:

- **Policy information.**
- **Policy holder information.**
- **Claim information.**
- **Claim status.**
- **Assignment information.**

2. Automation Layer

Salesforce Flow handles:

- **Record processing.**
- **Claim routing.**
- **Field updates.**
- **Business conditions.**

3. Apex Layer

Apex supports:

- **Advanced business logic.**
- **Complex processing.**
- **Server-side operations.**
- **Data retrieval and manipulation.**

4. LWC Layer

- **The Lightning Web Component provides the claims dashboard.**

5. Security Layer

Provides:

- **Object permissions.**
- **Field-level security.**
- **Record access.**
- **User/role-based access.**

5. PROJECT PLANNING AND SCHEDULING

5.1 Project Planning

The project was divided into four milestones.

Main Activities

Milestone 1

- Data model, policy objects, fields, relationships and policy configuration

Milestone 2

- Policy insurance automation, claim routing and business rules

Milestone 3

- LWC claims adjuster dashboard and filtering

Milestone 4

- Advanced claim processing, Apex, security, testing and code setup

Milestone	Focus	Planned Activities	Status
M1	Core Data Model & Policy Configuration	Data model, policy objects, fields, relationships and policy configuration	Completed
M2	Complex Policy Insurance & Claim Routing Automation	Policy/claim automation, routing conditions, assignment and business rules	Completed
M3	Claims Adjuster LWC Dashboard Development	LWC dashboard, claim display and client-side filtering	Completed
M4	Advanced Claim Processing, Security & Code Setup	Apex, advanced processing, security, validation and testing	Completed

Suggested Development Sequence

Planning → Data Model → Automation → LWC → Apex → Security → Testing → Final Documentation

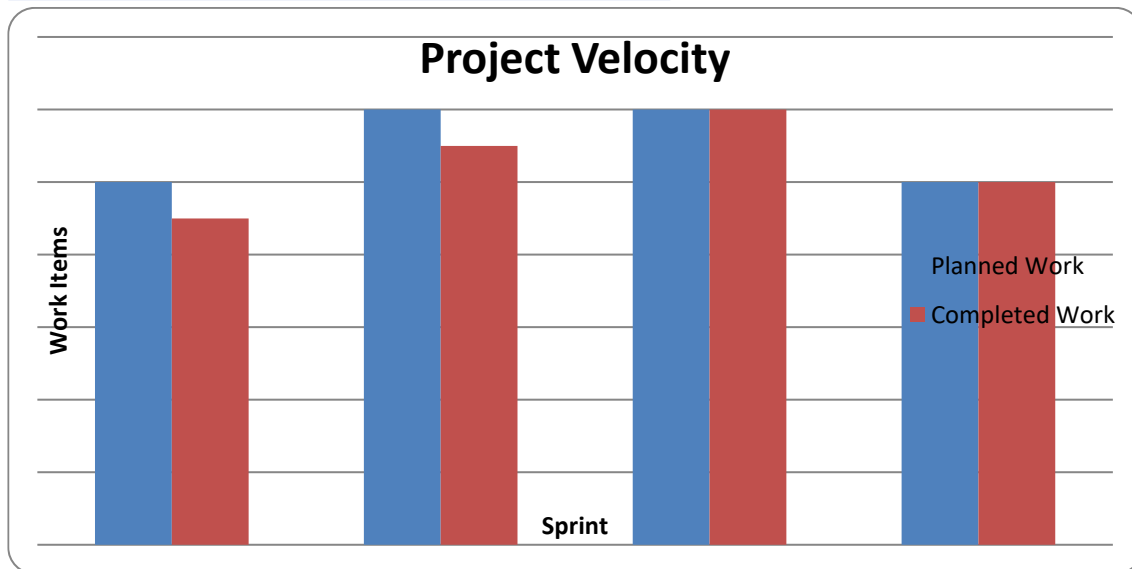
5.2 Project Tracker

Task ID	Task	Milestone	Status
T01	Define insurance requirements	M1	Completed
T02	Design data model	M1	Completed
T03	Configure policy information	M1	Completed
T04	Configure policy holder information	M1	Completed
T05	Configure claim structure	M1	Completed
T06	Implement claim routing	M2	Completed
T07	Implement automation	M2	Completed
T08	Configure claim assignment	M2	Completed
T09	Design LWC dashboard	M3	Completed
T10	Implement claim display	M3	Completed
T11	Implement filtering	M3	Completed
T12	Develop advanced processing	M4	Completed
T13	Implement Apex	M4	Completed
T14	Configure security	M4	Completed
T15	Perform testing	M4	Completed

5.3 Velocity

Velocity represents the amount of planned project work completed during a development period.

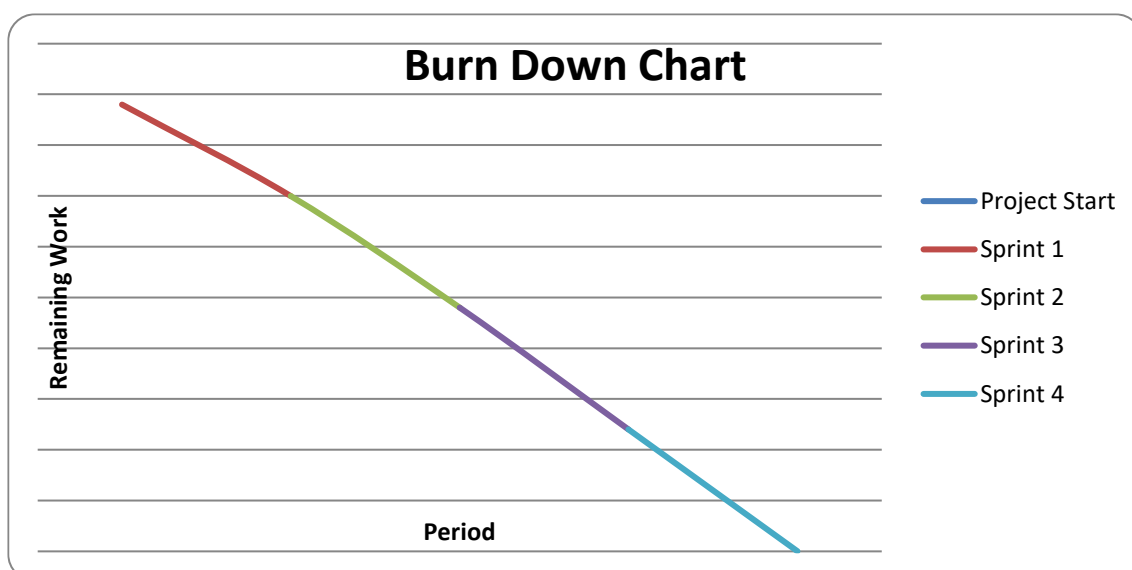
Sprint	Planned Work	Completed Work
Sprint 1	10	9
Sprint 2	12	11
Sprint 3	12	12
Sprint 4	10	10



5.4 Burn-Down Chart

A burn-down chart shows how much work remains during the project.

Period	Remaining Work
Project Start	44
Sprint 1	35
Sprint 2	24
Sprint 3	12
Sprint 4	0



6. PROJECT DEVELOPMENT PHASE

6.1 Development Activities

The development phase was completed through the four milestones.

Milestone 1

- **Core Data Model & Policy Configuration**
- **Development included:**
- **Creating insurance data structures.**
- **Creating required fields.**
- **Configuring policy information.**
- **Configuring policy holder information.**
- **Establishing record relationships.**
- **Preparing claim-related data.**

Milestone 2

Complex Policy Insurance & Claim Routing Automation

Development included:

- **Creating Salesforce Flows.**
- **Defining claim routing conditions.**
- **Automating claim processing.**
- **Updating claim fields.**
- **Assigning claims.**
- **Implementing business rules.**

Milestone 3

Claims Adjuster LWC Dashboard

Development included:

- **Creating Lightning Web Components.**
- **Connecting LWC to Apex.**
- **Displaying claim information.**
- **Implementing filtering.**
- **Creating an adjuster-friendly dashboard.**

Milestone 4

Advanced Claim Processing, Security & Code Setup

Development included:

- **Apex development.**
- **Advanced claim processing.**

- **Security configuration.**
- **Access control.**
- **Validation.**
- **Testing.**
- **Code organization.**

6.2 What You Will Learn

Through this project, the following skills are developed:

- **Salesforce Skills**
- **Salesforce object configuration.**
- **Custom fields.**
- **Relationships.**
- **Record management.**
- **Salesforce Flow.**
- **Security configuration.**
- **Programming Skills**
- **Apex.**
- **SOQL.**
- **JavaScript.**
- **HTML.**
- **CSS.**
- **LWC Skills**
- **Creating Lightning Web Components.**
- **Calling Apex methods.**
- **Using the wire service.**
- **Client-side filtering.**
- **Displaying Salesforce data.**
- **Project Skills**
- **Requirement analysis.**
- **System design.**
- **Data modeling.**
- **Agile planning.**
- **Testing.**
- **Documentation.**
- **Problem solving.**

Milestones

Milestone 1: Core Data Model & Policy Configuration

Creation of custom objects

1. Create Policy custom object:

- Go to setup, Click on object Manager.
- Click on create and select custom object.
- Fill the following details:
- Label: Policy
- Plural Label: Policies
- Object Name: Auto Populated
- Record Name: Policy Number
- Data type: Auto Number
- Format: P - {0000}
- Starting Number: 1
- Under optional features:
- Allow Reports, Activities and Track field History
- Under Deployment Status:
- Select the deployed radio button.
- Under Search Status:
- Check the Allow Search checkbox.
- Click on save.

The screenshot shows the Salesforce Setup interface for creating a new custom object. The browser tabs include WhatsApp, SkillWallet, Lightning Experience | Salesforce, New Custom Object | Salesforce, and an inbox. The URL is orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/new. The page title is 'New Custom Object'. A message states: 'Permissions for this object are disabled for all profiles by default. You can enable object permissions in permission sets or by editing custom profiles. Tell me more! Don't show this message again.' The 'Custom Object Definition Edit' section has buttons for 'Save', 'Save & New', and 'Cancel'. The 'Custom Object Information' section includes fields for 'Label' (Policy), 'Plural Label' (Policies), 'Object Name' (Policy), and 'Description'. Examples provided are 'Account' for Label and Object Name, and 'Accounts' for Plural Label. A 'Starts with vowel sound' checkbox is present. At the bottom, there are links for 'Context Sensitive Help Setting' and 'Open the standard Salesforce.com Help & Training window'.

WhatsApp x SkillWallet x Lightning Experience | Salesforce x New Custom Object | Salesforce x Inbox (176) - deborashalin12 x + Ask Gemini x

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/new

Developer Edition

Setup Home Object Manager

SETUP New Custom Object

Enter Record Name Label and Format

The Record Name appears in page layouts, key lists, related lists, lookups, and search results. For example, the Record Name for Account is "Account Name" and for Case it is "Case Number". Note that the Record Name field is always called "Name" when referenced via the API.

Record Name: Example: Account Name

Data Type: Warning: If you plan to insert a high volume of records in this object, via the API for example, use the Text data type.

Display Format: Example: A-(0000) What Is This?

Starting Number:

Optional Features

- ☒ Allow Reports
- ☒ Allow Activities
- ☒ Track Field History
- ☐ Allow in Chatter Groups
- ☐ Enable Licensing

Object Classification

When these settings are enabled, this object is classified as an Enterprise Application object. When these settings are disabled, this object is classified as a Light Application object. [Learn more.](#)

- ☒ Allow Sharing
- ☒ Allow Bulk API Access
- ☒ Allow Streaming API Access

Deployment Status

☐ In Development

What is this?

WhatsApp x SkillWallet x Lightning Experience | Salesforce x New Custom Object | Salesforce x Inbox (176) - deborashalin12 x + Ask Gemini x

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/new

Developer Edition

Setup Home Object Manager

SETUP New Custom Object

☐ Allow in Chatter Groups

☐ Enable Licensing

Object Classification

When these settings are enabled, this object is classified as an Enterprise Application object. When these settings are disabled, this object is classified as a Light Application object. [Learn more.](#)

- ☒ Allow Sharing
- ☒ Allow Bulk API Access
- ☒ Allow Streaming API Access

Deployment Status

☐ In Development

☒ Deployed

What is this?

Search Status

When this setting is enabled, your users can find records of this object type when they search. [Learn more.](#)

- ☒ Allow Search

Object Creation Options (Available only when custom object is first created)

- ☐ Add Notes and Attachments related list to default page layout
- ☐ Launch New Custom Tab Wizard after saving this custom object

Save Save & New Cancel

Define Record Types on Policy: Auto, Property, and Life.

- Go to setup, Click on Object Manager.
- Search for Policy object and click on Policy__c.
- In the left side panel, click on the record types.
- Click on New.
- Fill out the following details:
- Select the existing record type as Master.
- Give record type label as “Auto”
- Record Type Name will be auto populated.
- Give a meaningful description.
- Check the active checkbox.
- Check the Make Available checkbox for all the profiles.
- Click on Next.
- It will redirect to the Assign page layouts page, Leave everything as is, and click on save.

The screenshot shows the Salesforce Setup interface for defining a new Record Type. The browser address bar shows the URL: `orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01lgK000007gDeM/RecordTypes/new`. The left sidebar shows the navigation menu with 'Record Types' selected under the 'Policy' object. The main content area is titled 'New Record Type Policy' and shows 'Step 1. Enter the details'. The form fields are as follows:

Record Type	
Existing Record Type	--Master--
Record Type Label	Auto
Record Type Name	Auto
Description	
Active	☒

Below the form, there is a note: "Select Make Available to give users assigned to this profile the ability to create and clone records of this record type, or assign this record type to existing records. To make the new record type the default for a profile, select Make Default. Users assigned to this record type can still view and edit records associated with record types not available for their profiles."

At the bottom, there are checkboxes for 'Make Available' (checked) and 'Make Default' (unchecked).

WhatsApp x SkillWallet x Lightning Experience | Salesf... x Policy | Salesforce x Inbox (176) - deborashalini12 x + Ask Gemini

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01lgK000007gDeM/RecordTypes/012gK000006AfqL/view

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER
Policy

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Search Layouts
List View Button Layout

Record Type
Auto
« Back to Custom Object: Policy

Use the Edit button to change the properties of this record type. Use the Edit links in the Picklist Values related list to choose the picklist values available for records with this record type.

Edit

Record Type Label	Auto	Active	✓
Record Type Name	Auto		
Namespace Prefix			
Description			
Created By	Debora Shalini T	9/30/2026, 6:56 AM	Modified By Debora Shalini T 9/30/2026, 6:56 AM

26°C Clear 00:01:31 30-09-2026

WhatsApp x SkillWallet x Lightning Experience | Salesf... x Policy | Salesforce x Inbox (176) - deborashalini12 x + Ask Gemini

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01lgK000007gDeM/RecordTypes/new

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER
Policy

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Search Layouts
List View Button Layout

New Record Type
Policy

Step 1. Enter the details Step 1 of 2

Enter a name and description for the new record type. The new record type will include all the picklist values from the existing record type selected below. After saving the new record type, you will be able to customize the picklist values.

Record Type

Existing Record Type --Master--

Record Type Label Property

Record Type Name Property

Description

Active ☒

Select Make Available to give users assigned to this profile the ability to create and clone records of this record type, or assign this record type to existing records. To make the new record type the default for a profile, select Make Default. Users assigned to this record type can still view and edit records associated with record types not available for their profiles.

Profile Name Record Types Currently Available ☒ Make Available ☐ Make Default

26°C Clear 00:02:38 30-09-2026

Screenshot of the Salesforce Setup page for creating a new Record Type. The browser tabs include WhatsApp, SkillWallet, Lightning Experience | Salesforce, Policy | Salesforce, and Inbox (176) - deborashalin12. The URL is orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01gK000007gDeM/RecordTypes/new. The page is titled "Policy" and shows the "New Record Type" setup process. The left sidebar lists navigation options: Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types (selected), Related Lookup Filters, Search Layouts, and List View Button Layout. The main content area is titled "Step 1. Enter the details" and includes a "Record Type" section with the following fields: Existing Record Type (Master), Record Type Label (Property), Record Type Name (Property), Description, and Active (checked). A "Required Information" icon is present. Below the fields, there is a section for "Profile Name" and "Record Types Currently Available" with checkboxes for "Make Available" and "Make Default". The bottom status bar shows 26°C Clear and a system clock of 00:02:38 on 30-09-2026.

Screenshot of the Salesforce Setup page for viewing a Record Type. The browser tabs include WhatsApp, SkillWallet, Lightning Experience | Salesforce, Policy | Salesforce, and Inbox (176) - deborashalin12. The URL is orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01gK000007gDeM/RecordTypes/012gK000006AFrx/view. The page is titled "Policy" and shows the "Record Type" setup process. The left sidebar lists navigation options: Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types (selected), Related Lookup Filters, Search Layouts, and List View Button Layout. The main content area is titled "Record Type Property" and includes a "Back to Custom Object: Policy" link. Below the title, there is an "Edit" button. The main content area displays the following information: Record Type Label (Property), Record Type Name (Property), Namespace Prefix, Description, Created By (Debora Shalini T, 9/30/2026, 6:57 AM), Modified By (Debora Shalini T, 9/30/2026, 6:57 AM), and Active (checked). The bottom status bar shows 26°C Clear and a system clock of 00:02:57 on 30-09-2026.

WhatsApp x SkillWallet x Lightning Experience | Salesf... x Policy | Salesforce x Inbox (176) - deborashalini12 x + Ask Gemini

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01gK000007gDeM/RecordTypes/new

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER

Policy

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

New Record Type

Policy

Step 1 of 2

Step 1. Enter the details

Enter a name and description for the new record type. The new record type will include all the picklist values from the existing record type selected below. After saving the new record type, you will be able to customize the picklist values.

Record Type

Existing Record Type: --Master--

Record Type Label: Life

Record Type Name: Life

Description:

Active: ☒

Select Make Available to give users assigned to this profile the ability to create and clone records of this record type, or assign this record type to existing records. To make the new record type the default for a profile, select Make Default. Users assigned to this record type can still view and edit records associated with record types not available for their profiles.

Profile Name: Record Types Currently Available: ☒ Make Available ☐ Make Default

26°C Clear

00:03:20

WhatsApp x SkillWallet x Lightning Experience | Salesf... x Policy | Salesforce x Inbox (176) - deborashalini12 x + Ask Gemini

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01gK000007gDeM/RecordTypes/012gK000006AfZ/view

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER

Policy

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

Record Type

Life

< Back to Custom Object: Policy

Use the Edit button to change the properties of this record type. Use the Edit links in the Picklist Values related list to choose the picklist values available for records with this record type.

Edit

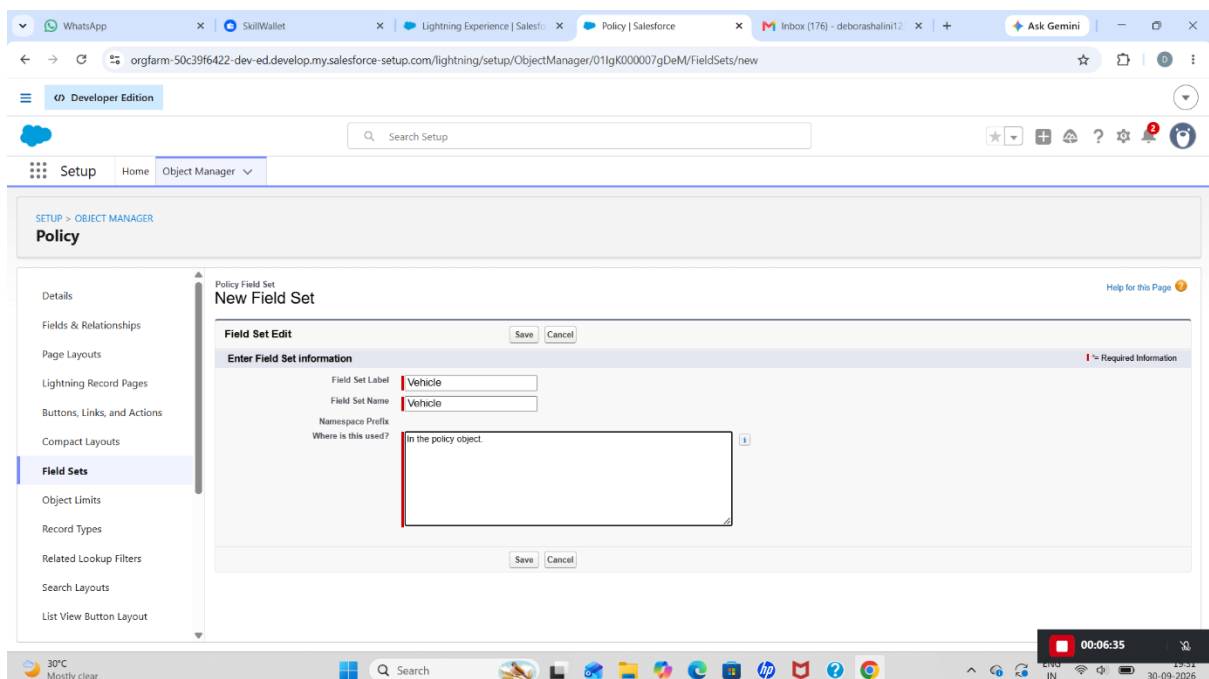
Record Type Label	Life	Active	☒
Record Type Name	Life		
Namespace Prefix			
Description			
Created By	Debora Shalini T	9/30/2026, 6:58 AM	Modified By: Debora Shalini T, 9/30/2026, 6:58 AM

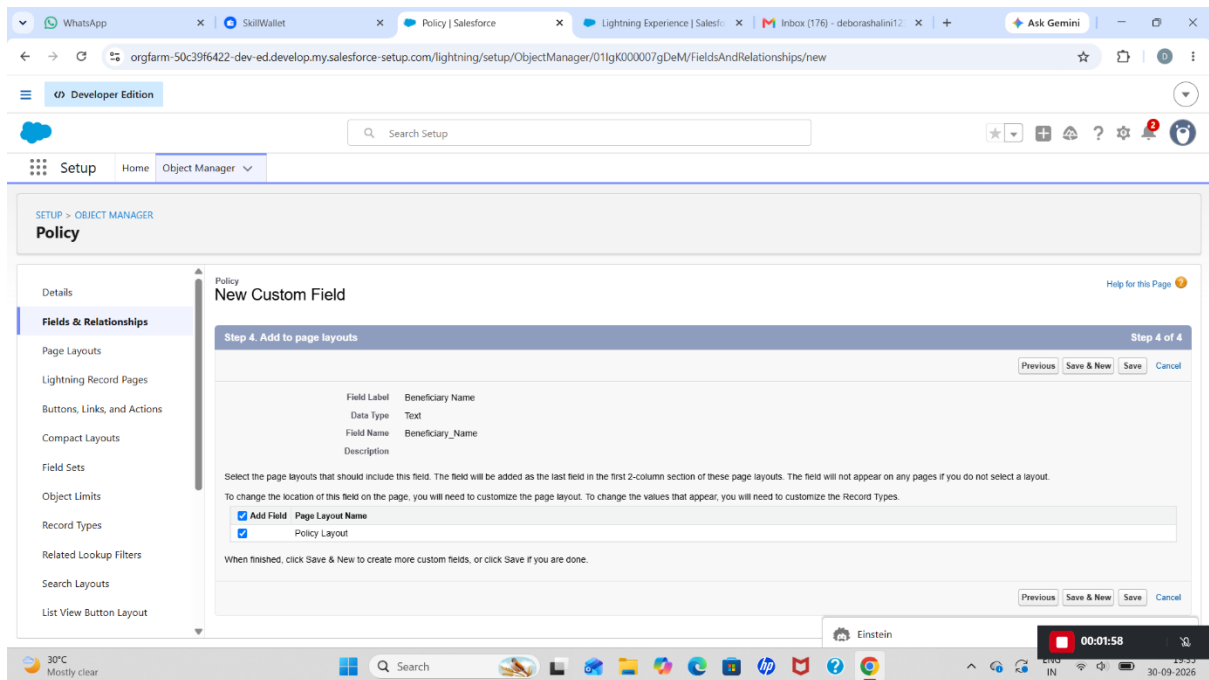
30°C Mostly clear

00:03:37

Create Field Sets for policy-specific data (e.g., Auto_Fields requires VIN and Model Year; Property_Fields requires Square Footage).

- Go to setup, Click on Object Manager.
- Search for Policy object and click on Policy__c.
- In the left side panel, click on the Field Sets.
- Click on New.
- Fill out the following details:
- Field Set Label as Vehicle
- Field Set Name will be Auto populated.
- Where is this used: In the policy object.
- Click on save.
- Click on the field set you have just created and it will redirect to the page layout setup.
- Drag VIN, and model year fields to the field set and click on save.





Repeat the same steps for creating the property and life Field sets as well.

3.1 Property FieldSet Requires:

Square footage

Year Built

3.2 Life FieldSet Requires:

Beneficiary Name

Policy Term Months

Create Fields in the Policy Object

S.No	Name of the Field	Data Type	Required
1	Beneficiary Name	Text(50)	
2	Customer	Lookup(Contact)	
3	Model Year	Text(5)	
4	Policy Start Date	Date	
5	Policy Term Months	Number(18, 0)	
6	Premium	Currency(16, 2)	
7	Square Footage	Number(18, 0)	
8	VIN	Text(20)	
9	Year Built	Text(5)	

The screenshot displays the Salesforce Developer Edition interface for configuring a new relationship for the 'Policy' object. The left sidebar contains navigation links: Details, Fields & Relationships (active), Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, Search Layouts, and List View Button Layout. The main content area is titled 'New Relationship' and shows 'Step 5 of 6: Add reference field to Page Layouts'. A table summarizes the field configuration:

Field Label	Customer
Data Type	Lookup
Field Name	Customer
Description	

Below the table, instructions state: 'Select the page layouts that should include this field. The field will be added as the last field in the first 2-column section of these page layouts. The field will not appear on any pages if you do not select a layout. To change the location of this field on the page, you will need to customize the page layout. To change the values that appear, you will need to customize the Record Types.' A section titled 'Add Field' shows 'Page Layout Name' with 'Policy Layout' selected. Navigation buttons 'Previous', 'Next', and 'Cancel' are visible at the bottom of the step.

WhatsApp SkillWallet Policy | Salesforce Lightning Experience | Salesf Inbox (176) - deborashini12 Ask Gemini

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01gK000007gDeM/FieldsAndRelationships/new

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER Policy

Details Fields & Relationships Page Layouts Lightning Record Pages Buttons, Links, and Actions Compact Layouts Field Sets Object Limits Record Types Related Lookup Filters Search Layouts List View Button Layout

Policy New Custom Field

Step 4. Add to page layouts

Field Label Model Year
Data Type Text
Field Name Model_Year
Description

Select the page layouts that should include this field. The field will be added as the last field in the first 2-column section of these page layouts. The field will not appear on any pages if you do not select a layout.

To change the location of this field on the page, you will need to customize the page layout. To change the values that appear, you will need to customize the Record Types.

☒ Add Field Page Layout Name
☒ Policy Layout

When finished, click Save & New to create more custom fields, or click Save if you are done.

Previous Save & New Save Cancel

Einstein 00:04:45 30-09-2026

WhatsApp SkillWallet Policy | Salesforce Lightning Experience | Salesf Inbox (176) - deborashini12 Ask Gemini

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01gK000007gDeM/FieldsAndRelationships/new

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER Policy

Details Fields & Relationships Page Layouts Lightning Record Pages Buttons, Links, and Actions Compact Layouts Field Sets Object Limits Record Types Related Lookup Filters Search Layouts List View Button Layout

Policy New Custom Field

Step 4. Add to page layouts

Field Label Policy Start Date
Data Type Date
Field Name Policy_Start_Date
Description

Select the page layouts that should include this field. The field will be added as the last field in the first 2-column section of these page layouts. The field will not appear on any pages if you do not select a layout.

To change the location of this field on the page, you will need to customize the page layout. To change the values that appear, you will need to customize the Record Types.

☒ Add Field Page Layout Name
☒ Policy Layout

When finished, click Save & New to create more custom fields, or click Save if you are done.

Previous Save & New Save Cancel

Einstein 00:05:37 30-09-2026

WhatsApp x SkillWallet x Policy | Salesforce x Lightning Experience | Salesf x Inbox (176) - deborashini12 x Ask Gemini x

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01gK000007gDeM/FieldsAndRelationships/new

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER

Policy

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

Policy

New Custom Field

Step 4. Add to page layouts

Previous Save & New Save Cancel

Field Label	Policy Term Months
Data Type	Number
Field Name	Policy_Term_Months
Description	

Select the page layouts that should include this field. The field will be added as the last field in the first 2-column section of these page layouts. The field will not appear on any pages if you do not select a layout.

To change the location of this field on the page, you will need to customize the page layout. To change the values that appear, you will need to customize the Record Types.

☒ Add Field Page Layout Name

☒ Policy Layout

When finished, click Save & New to create more custom fields, or click Save if you are done.

Previous Save & New Save Cancel

Einstein

00:06:18

30°C Mostly clear

Search

30-09-2026

WhatsApp x SkillWallet x Policy | Salesforce x Lightning Experience | Salesf x Inbox (176) - deborashini12 x Ask Gemini x

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01gK000007gDeM/FieldsAndRelationships/new

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER

Policy

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

Policy

New Custom Field

Step 4. Add to page layouts

Previous Save & New Save Cancel

Field Label	Premium
Data Type	Currency
Field Name	Premium
Description	

Select the page layouts that should include this field. The field will be added as the last field in the first 2-column section of these page layouts. The field will not appear on any pages if you do not select a layout.

To change the location of this field on the page, you will need to customize the page layout. To change the values that appear, you will need to customize the Record Types.

☒ Add Field Page Layout Name

☒ Policy Layout

When finished, click Save & New to create more custom fields, or click Save if you are done.

Previous Save & New Save Cancel

Einstein

00:08:06

30°C Mostly clear

Search

30-09-2026

WhatsApp SkillWallet Policy | Salesforce Lightning Experience | Salesf Inbox (176) - deborashini12 Ask Gemini

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01gK000007gDeM/FieldsAndRelationships/new

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER Policy

Details Fields & Relationships Page Layouts Lightning Record Pages Buttons, Links, and Actions Compact Layouts Field Sets Object Limits Record Types Related Lookup Filters Search Layouts List View Button Layout

Policy New Custom Field

Step 4. Add to page layouts Step 4 of 4

Field Label Square Footage
Data Type Number
Field Name Square_Footage
Description

Select the page layouts that should include this field. The field will be added as the last field in the first 2-column section of these page layouts. The field will not appear on any pages if you do not select a layout.

To change the location of this field on the page, you will need to customize the page layout. To change the values that appear, you will need to customize the Record Types.

☒ Add Field Page Layout Name
☒ Policy Layout

When finished, click Save & New to create more custom fields, or click Save if you are done.

Previous Save & New Save Cancel

30°C Mostly clear Search Einstein 00:08:55 30-09-2026

WhatsApp SkillWallet Policy | Salesforce Lightning Experience | Salesf Inbox (176) - deborashini12 Ask Gemini

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01gK000007gDeM/FieldsAndRelationships/new

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER Policy

Details Fields & Relationships Page Layouts Lightning Record Pages Buttons, Links, and Actions Compact Layouts Field Sets Object Limits Record Types Related Lookup Filters Search Layouts List View Button Layout

Policy New Custom Field

Step 4. Add to page layouts Step 4 of 4

Field Label VIN
Data Type Text
Field Name VIN
Description

Select the page layouts that should include this field. The field will be added as the last field in the first 2-column section of these page layouts. The field will not appear on any pages if you do not select a layout.

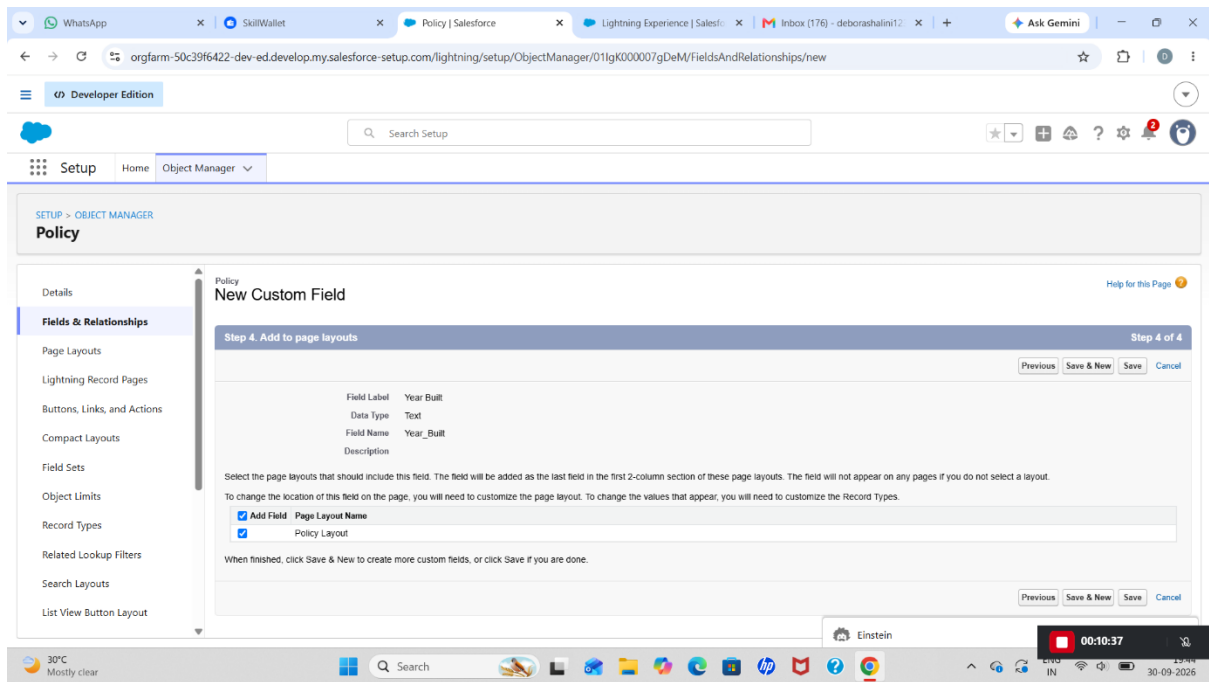
To change the location of this field on the page, you will need to customize the page layout. To change the values that appear, you will need to customize the Record Types.

☒ Add Field Page Layout Name
☒ Policy Layout

When finished, click Save & New to create more custom fields, or click Save if you are done.

Previous Save & New Save Cancel

30°C Mostly clear Search Einstein 00:09:46 30-09-2026



Create Claim custom object

- Go to setup, Click on object Manager.
- Click on create and select custom object.
- Fill the following details:
- Label: Claim
- Plural Label: Claim
- Object Name: Auto Populated
- Record Name: Claim Number
- Data type: Auto Number
- Format: C - {0000}
- Starting Number: 1
- Under optional features:
- Allow Reports, Activities and Track field History
- Under Deployment Status:
- Select the deployed radio button.
- Under Search Status:
- Check the Allow Search checkbox.
- Click on save.

WhatsApp SkillWallet New Custom Object | Salesforce Lightning Experience | Sales Inbox (176) - deborashalin12 Ask Gemini

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/new

Developer Edition

Setup Home Object Manager

SETUP New Custom Object

New Custom Object

Permissions for this object are disabled for all profiles by default. You can enable object permissions in permission sets or by editing custom profiles. [Tell me more!](#) [Don't show this message again](#)

Custom Object Definition Edit Save Save & New Cancel

Custom Object Information Required Information

The singular and plural labels are used in tabs, page layouts, and reports.

Label Example: Account

Plural Label Example: Accounts

Starts with vowel sound ☐

The Object Name is used when referencing the object via the API.

Object Name Example: Account

Description

Context-Sensitive Help Setting ☒ Open the standard Salesforce.com Help & Training window ☐ Open a window using a Visualforce page

30°C Mostly clear 00:19:20 30-09-2026

WhatsApp SkillWallet Claim | Salesforce Lightning Experience | Sales Inbox (176) - deborashalin12 Ask Gemini

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01lgK000007gGSX/RecordTypes/new

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER Claim

Details Fields & Relationships Page Layouts Lightning Record Pages Buttons, Links, and Actions Compact Layouts Field Sets Object Limits **Record Types** Related Lookup Filters Search Layouts List View Button Layout

New Record Type Claim

Step 1. Enter the details Step 1 of 2

Enter a name and description for the new record type. The new record type will include all the picklist values from the existing record type selected below. After saving the new record type, you will be able to customize the picklist values.

Record Type Required Information

Existing Record Type

Record Type Label

Record Type Name

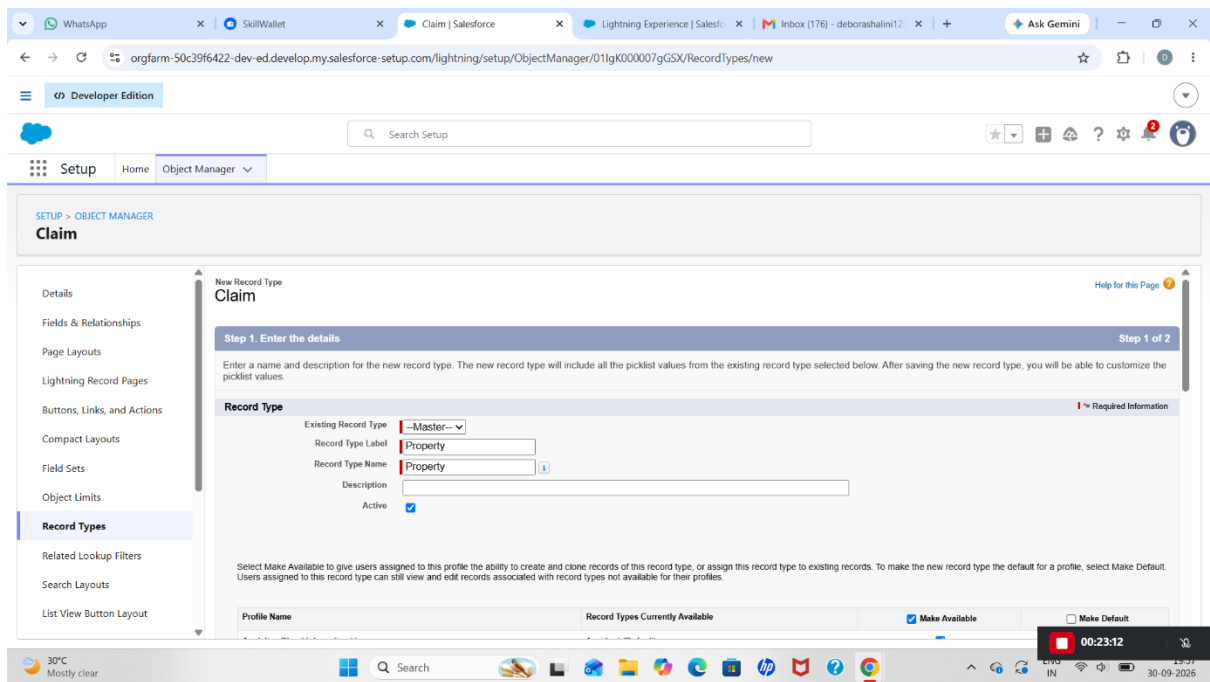
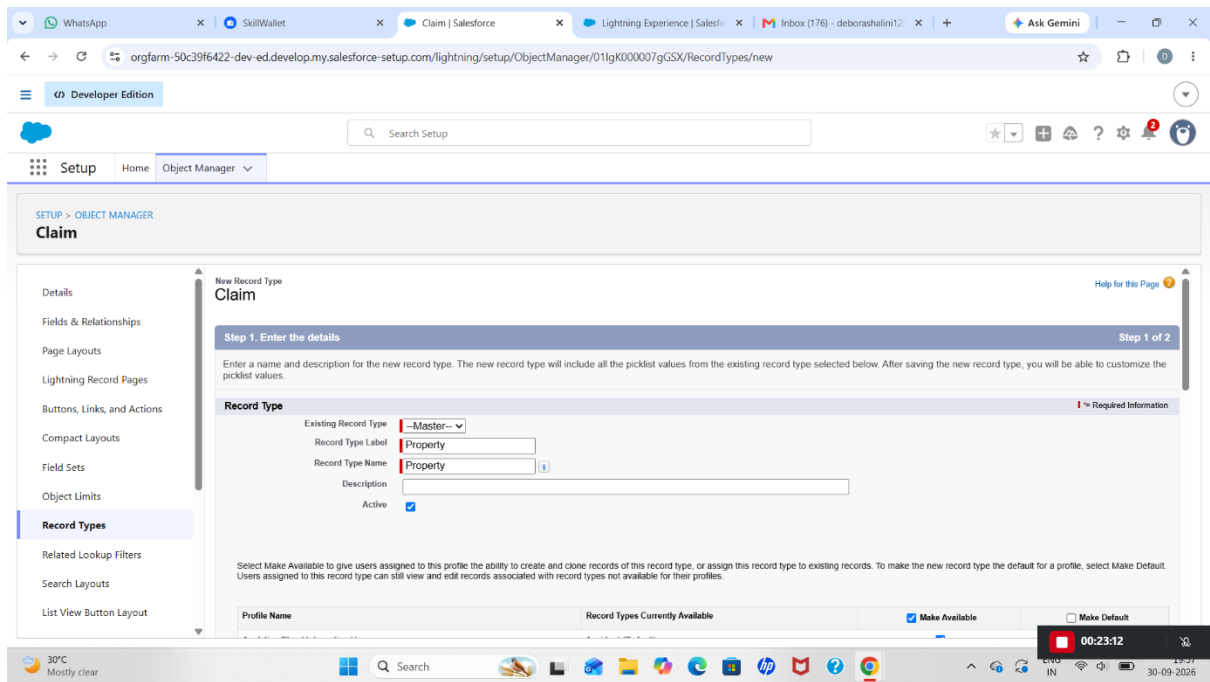
Description

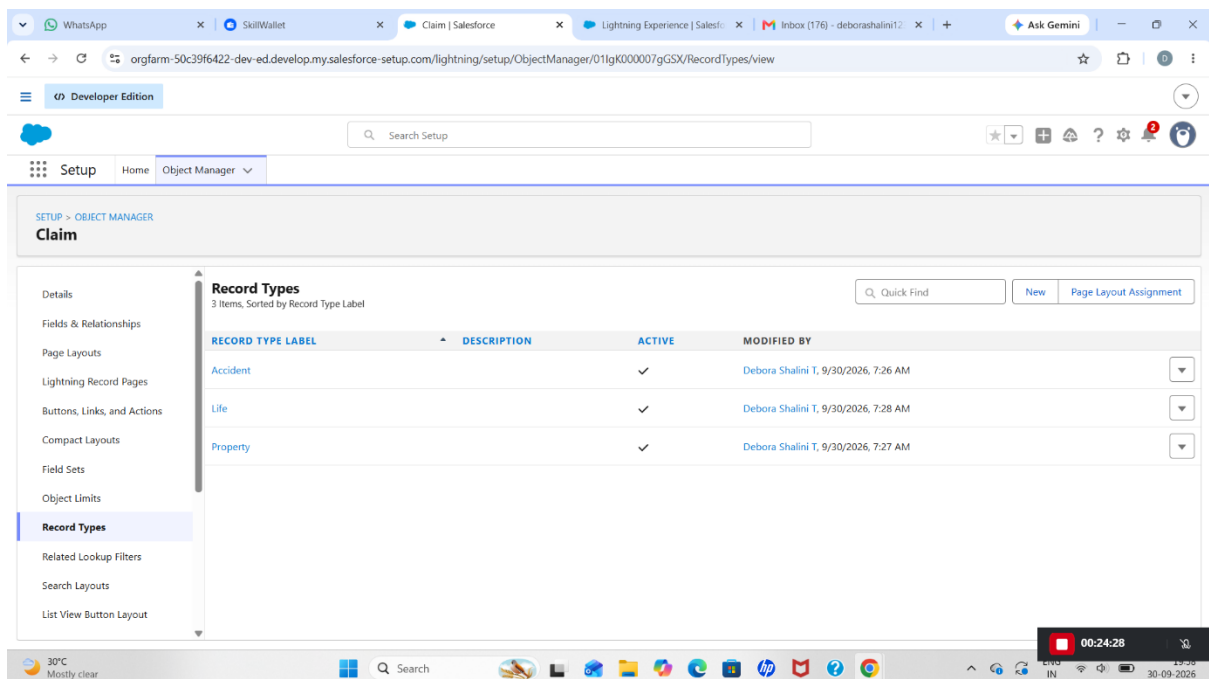
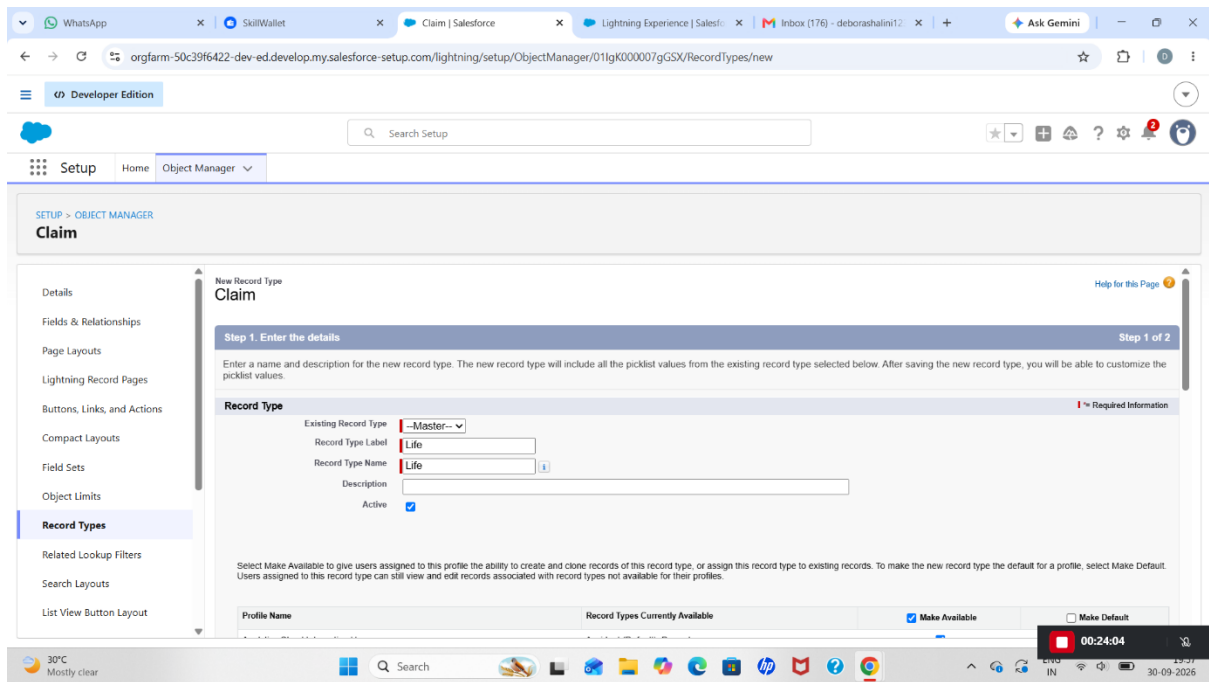
Active ☒

Select Make Available to give users assigned to this profile the ability to create and clone records of this record type, or assign this record type to existing records. To make the new record type the default for a profile, select Make Default. Users assigned to this record type can still view and edit records associated with record types not available for their profiles.

Profile Name Record Types Currently Available ☒ Make Available ☐ Make Default

30°C Mostly clear 00:21:56 30-09-2026





Define Record Types on Claim Object

- Go to setup, Click on Object Manager.
- Search for Claim object and click on Claim__c.
- In the left side panel, click on the record types.

- Click on New.
- Fill out the following details:
- Select the existing record type as Master.
- Give record type label as “Accident”
- Record Type Name will be auto populated.
- Give a meaningful description.
- Check the active checkbox.
- Check the Make Available checkbox for all the profiles.
- Click on Next.
- It will redirect to the Assign page layouts page, Leave everything as is, and click on save.

The screenshot shows the Salesforce Setup interface. The breadcrumb trail is **SETUP > OBJECT MANAGER > Policy**. The left sidebar shows the navigation menu with **Fields & Relationships** selected. The main content area is titled **New Relationship** and shows **Step 3. Enter the label and name for the lookup field** (Step 3 of 5). The form contains the following fields:

- Field Label:** Adjuster
- Field Name:** Adjuster
- Description:** (empty)
- Help Text:** (empty)
- Child Relationship Name:** Policies1
- Auto add to custom report type:** ☒ Add this field to existing custom report types that contain this entity

At the bottom, there is a **Lookup Filter** section. The browser's address bar shows the URL: `orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01lgK000007gDeM/FieldsAndRelationships/new`. The taskbar at the bottom shows the system clock as 00:26:04 on 30-09-2026.

WhatsApp SkillWallet Policy | Salesforce Lightning Experience | Salesf Inbox (176) - deborashini2 Ask Gemini

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01gK000007gDeM/FieldsAndRelationships/new

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER Policy

Details Fields & Relationships Page Layouts Lightning Record Pages Buttons, Links, and Actions Compact Layouts Field Sets Object Limits Record Types Related Lookup Filters Search Layouts List View Button Layout

Policy New Custom Field Step 2. Enter the details Step 2 of 4 Previous Next Cancel

Field Label Approval Status

Values ☐ Use global picklist value set ☒ Enter values, with each value separated by a new line

New Submitted for Approval Approved Rejected

☐ Display values alphabetically, not in the order entered ☐ Use first value as default value ☒ Restrict picklist to the values defined in the value set

Field Name Approval_Status

30°C Mostly clear Einstein 00:27:40 30-09-2026

WhatsApp SkillWallet Policy | Salesforce Lightning Experience | Salesf Inbox (176) - deborashini2 Ask Gemini

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01gK000007gDeM/FieldsAndRelationships/new

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER Policy

Details Fields & Relationships Page Layouts Lightning Record Pages Buttons, Links, and Actions Compact Layouts Field Sets Object Limits Record Types Related Lookup Filters Search Layouts List View Button Layout

Policy New Custom Field Step 2. Enter the details Step 2 of 4 Previous Next Cancel

Field Label Claim Amount

Please enter the length of the number and the number of decimal places. For example, a number with a length of 8 and 2 decimal places can accept values up to "12345678.90".

Length 16 Decimal Places 2

Number of digits to the left of the decimal point Number of digits to the right of the decimal point

Field Name Claim_Amount

Description

Help Text

Required ☐ Always require a value in this field in order to save a record

Auto add to custom report type ☒ Add this field to existing custom report types that contain this entity

Einstein 00:28:50 30-09-2026

WhatsApp SkillWallet Policy | Salesforce Lightning Experience | Salesf Inbox (176) - deborashini12 Ask Gemini Close

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01gK000007gDeM/FieldsAndRelationships/new

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER Policy

Details Fields & Relationships Page Layouts Lightning Record Pages Buttons, Links, and Actions Compact Layouts Field Sets Object Limits Record Types Related Lookup Filters Search Layouts List View Button Layout

Policy New Custom Field Step 2. Enter the details Step 2 of 4 Previous Next Cancel

Field Label Date of Loss

Field Name Date of Loss

Description

Help Text

Required ☐ Always require a value in this field in order to save a record

Auto add to custom report type ☒ Add this field to existing custom report types that contain this entity

Default Value Show Formula Editor

Use formula syntax. Enclose text and picklist value API names in double quotes ("the_text"). Include members without quotes.

30°C Mostly clear

Search

Einstein 00:29:50

30-09-2026

WhatsApp SkillWallet Policy | Salesforce Lightning Experience | Salesf Inbox (176) - deborashini12 Ask Gemini Close

orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01gK000007gDeM/FieldsAndRelationships/new

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER Policy

Details Fields & Relationships Page Layouts Lightning Record Pages Buttons, Links, and Actions Compact Layouts Field Sets Object Limits Record Types Related Lookup Filters Search Layouts List View Button Layout

Policy New Custom Field Step 2. Enter the details Step 2 of 4 Previous Next Cancel

Field Label Description

Length 32,768 (Max 131,072)

Visible Lines 3

Field Name Description

Description

Help Text

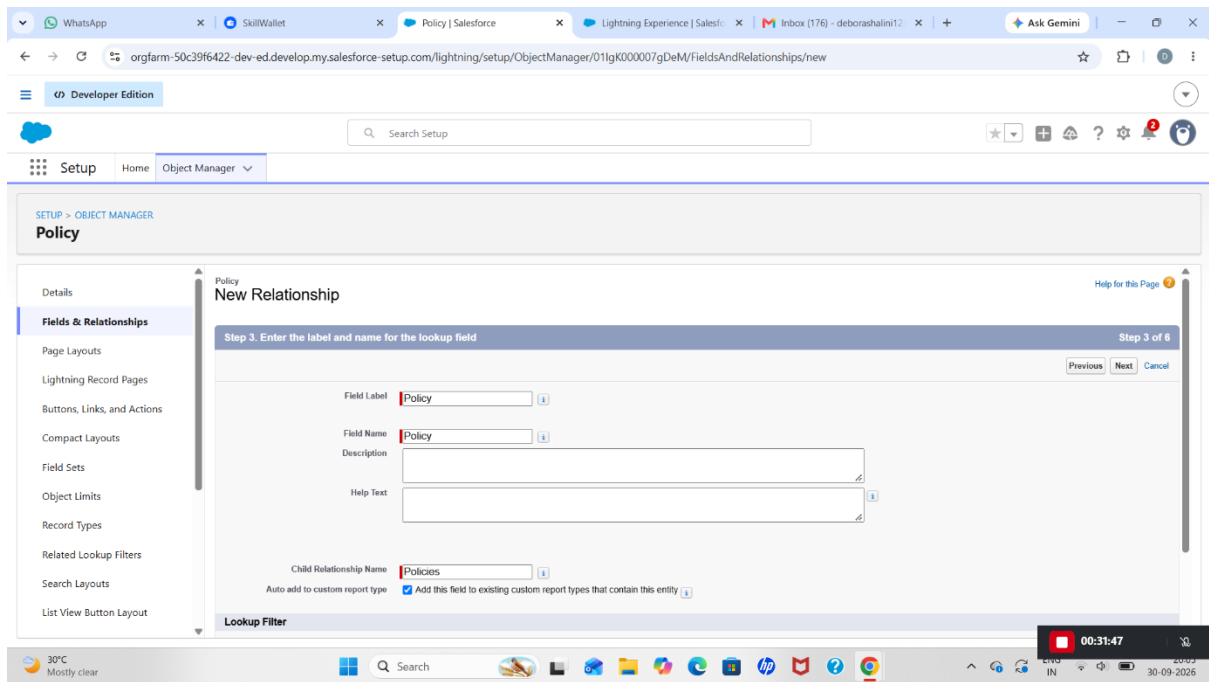
You are currently using 0 out of 1,638,400 characters on this object. You have 1,638,400 additional characters to allocate to this field.

30°C Mostly clear

Search

Einstein 00:30:47

30-09-2026



Repeat the same steps for creating the property and life record types as well.

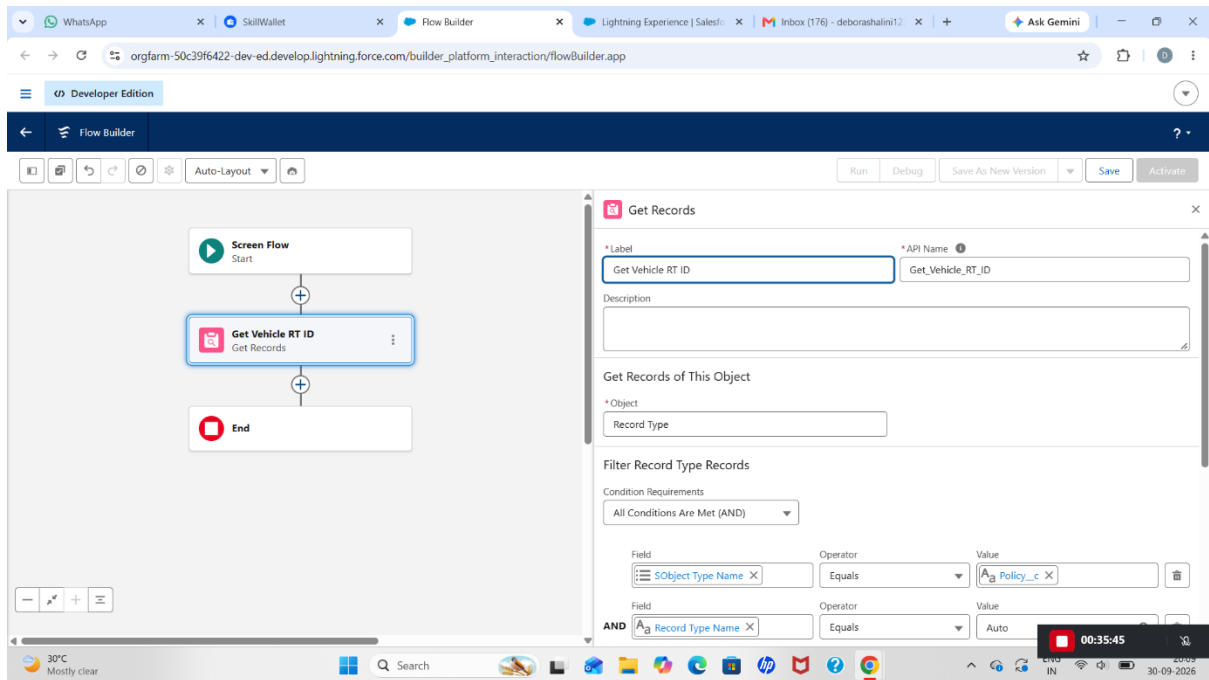
Create Fields in the Policy Object

S.No	Field Name	Data Type	Required
1	Adjuster	Lookup(User)	
2	Approval Status	Picklist(New, Submitted for Approval, Approved, Rejected)	
3	Claim Amount	Currency(16, 2)	
4	Date of Loss	Date/Time	
5	Description	Long Text Area	
6	Policy	Lookup(Policy)	

Create an AutoQuoting Screen Flow

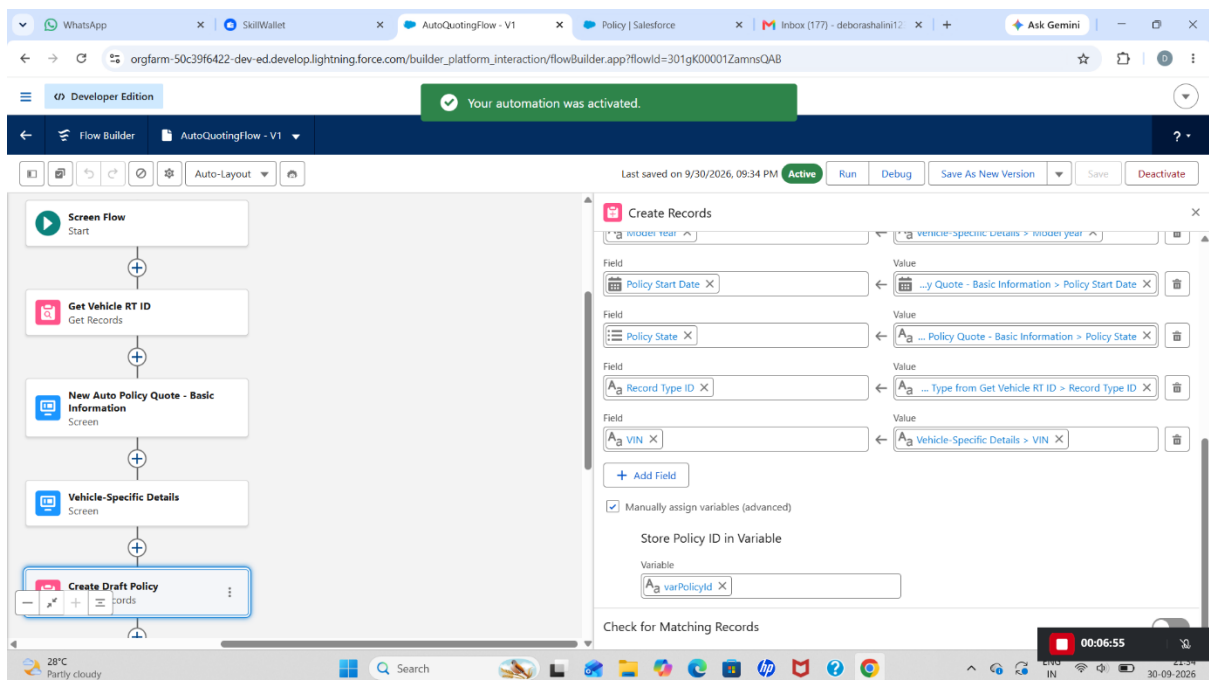
- Go to setup, Search for flows in the quick find box.
- Click on flows and click on new flow.
- Select Screen Flow and Click on Create.
- Add Get Records Element immediately after the start element.
- Give Label as Get Vehicle RT ID
- Api Name will be auto populated
- Select Record Type for Object
- For Conditional Requirements, Select All Conditions are met

- **The Conditions include:**
- **sObject Type Name Equals Policy__c**
- **And**
- **Record Type Name Equals Auto**



- **Add Screen Element immediately after the Get Records Element**
- **Give the label as New Auto Policy Quote - Basic Information**
- **API Name will be auto populated.**
- **Drag the display text component into the flow canvas**
- **Give the API Name as Screen_1_Policy_Basics**
- **Drag the Lookup Component into the Canvas, Give API Name as Customer Lookup, Field API Name as Customer__c, Label as Policy Holder, ObjectApiName as Policy__c**
- **Drag Date component into the canvas, Give Label as Policy Start Date, API Name will be auto populated.**
- **Drag the picklist component into the canvas, Give the label as Policy State, API Name will be auto populated, For Choice, Create a new Resource, Resource Type should be Picklist Choice Set, Give the API Name, Select the Policy for Object, Select Picklist as Datatype, Select Policy State as Field and Click on save**
- **Click on Done**

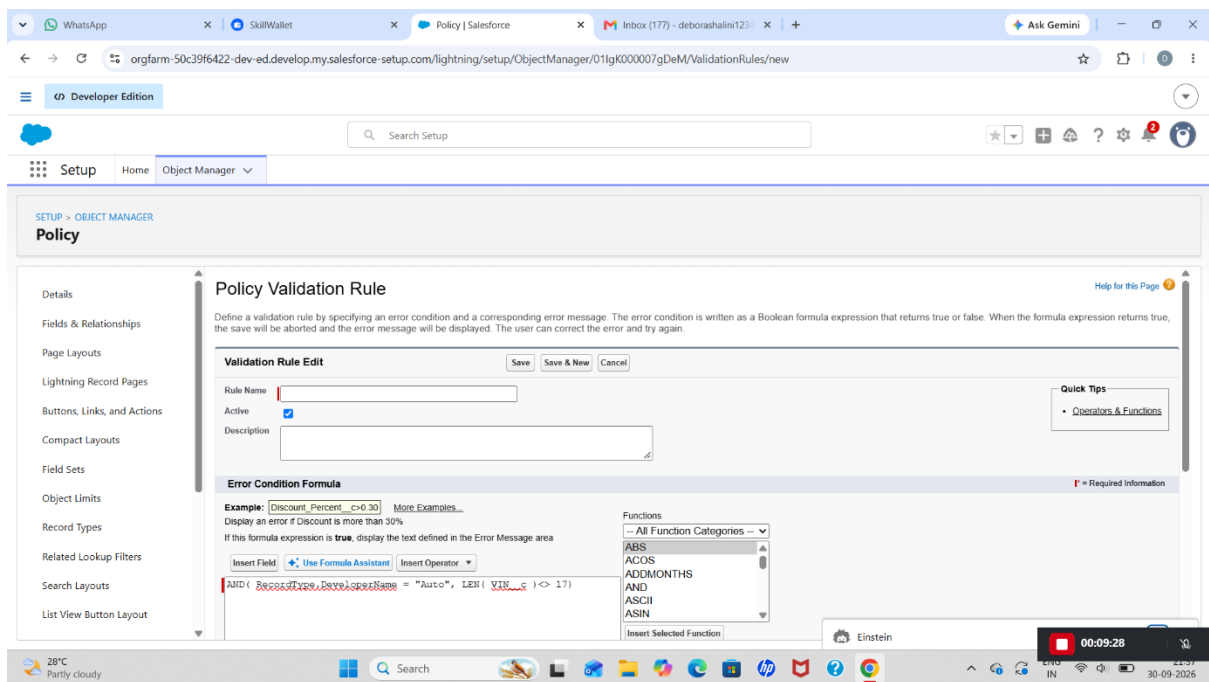
- Add another screen element after the Auto Policy Quote - Basic Information screen
- Give the label as Vehicle-Specific Details
- API Name will be auto populated.
- Drag the display text component into the flow canvas
- Give the API Name as Screen_2_Vehicle_Details
- Drag the text component into the canvas, Give the label as VIN, API name will be auto populated
- Again, Drag the text component into the canvas, Give the label as Model year, API name will be auto populated.
- Click on done
- Add Create Records Element after the screen element.
- Give the label as Create Draft Policy, API name will be auto populated
- Store field values manually
- Select the policy object
- Set fields for the policy object as shown below:
- Customer —> From Basic information screen-> Policy Holder -> Record ID
- Model Year——> Vehicle information screen -> Model Year
- Policy Start Date —> From Basic information screen→Policy Start Date
- Policy State —> From Basic information screen —> Policy State
- RecordType ID —> From Get Vehicle RT Id —> RecordType ID
- VIN —> From Vehicle Information Screen —> VIN
- Check manually assign variables checkbox
- In the place of variable, create a new resource, Whose resource type is variable, API Name would be varPolicyId, Datatype would be Text, Click on Done.



- Save the flow with AutoQuotingFlow
- Activate the flow

Create Validation Rule on Policy Object

- Go to object manager, Search for the policy object and click on policy object
- In the left side panel, scroll down and find validation rules and click on it
- Click on New



- Check the active checkbox
- Write the following formula in the formula editor
- **AND(RecordType.DeveloperName = "Auto", LEN(VIN__c) <> 17)**
- For error message, The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.
- Select error location as field and select VIN field
- Click on Save

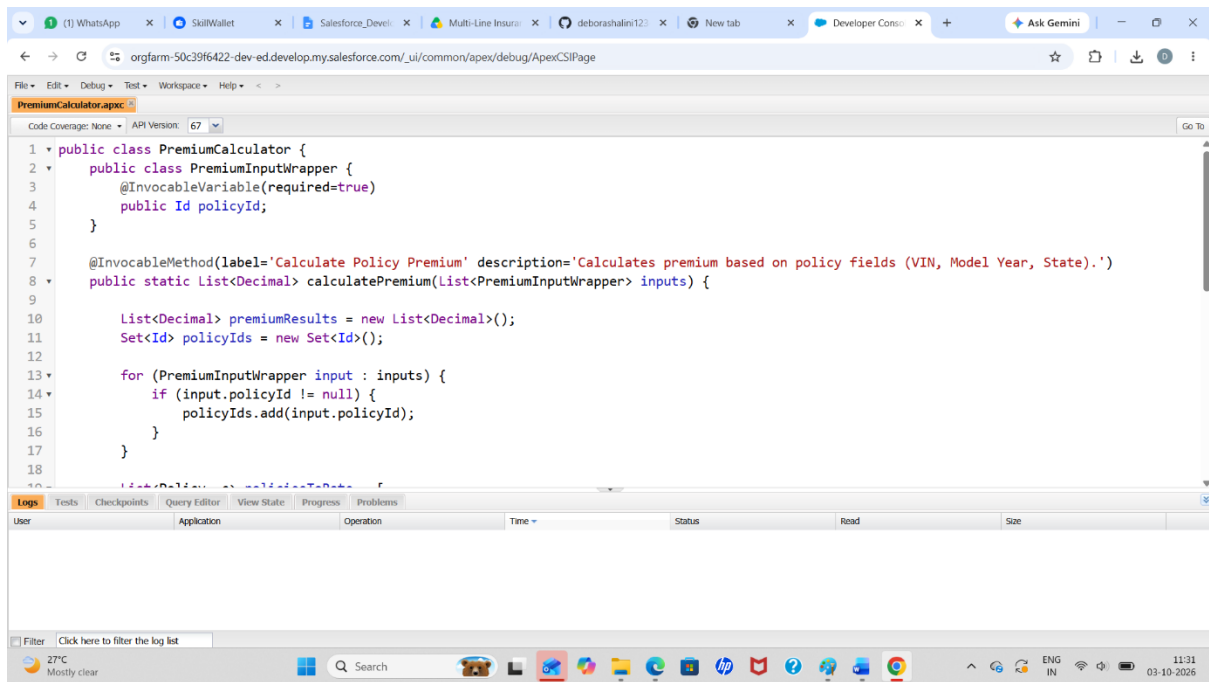
Milestone 2: Complex Policy Insurance & Claim Routing Automation

Create an Apex Class (PremiumCalculator) with a method that accepts policy data

- Click on setup icon, Right click on developer console, Open the link in new tab
- At the top left corner, click on the file, then click on new, and click on Apex class, Give the name as PremiumCalculator and click on create.
- Paste the following code in the apex class:

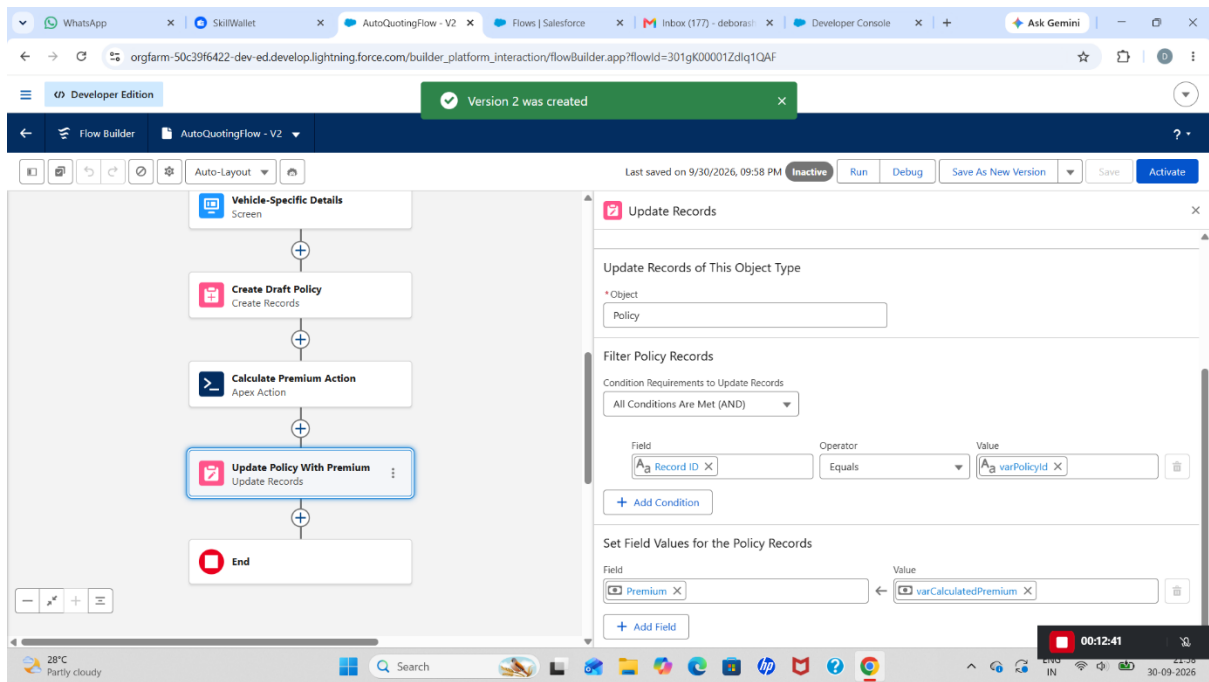
```
public class PremiumCalculator { public class PremiumInputWrapper {  
@InvocableVariable(required=true) public Id policyId; }  
@InvocableMethod(label='Calculate Policy Premium' description='Calculates premium  
based on policy fields (VIN, Model Year, State).') public static List<Decimal>  
calculatePremium(List<PremiumInputWrapper> inputs) { List<Decimal>  
premiumResults = new List<Decimal>(); Set<Id> policyIds = new Set<Id>(); for  
(PremiumInputWrapper input : inputs) { if (input.policyId != null) {  
policyIds.add(input.policyId); } } List<Policy__c> policiesToRate = [ SELECT Id,  
VIN__c, Model_Year__c, Policy_State__c FROM Policy__c WHERE Id IN :policyIds ];  
for (Policy__c policy : policiesToRate) { Decimal basePremium = 1000.00; if  
(policy.Policy_State__c == 'CA') { basePremium *= 1.15; } else if (policy.Policy_State__c  
== 'TX') { basePremium *= 1.05; } if (policy.Model_Year__c != null &&  
Integer.valueOf(policy.Model_Year__c) < 2018) { basePremium += 100; }  
premiumResults.add(basePremium); } return premiumResults; } }
```

- Click on CTRL + S to save the apex class
- And this apex class need to be invoked in the AutoQuotingFlow



Integrate this Apex action into the final step of the AutoQuotingFlow to update the draft Policy record

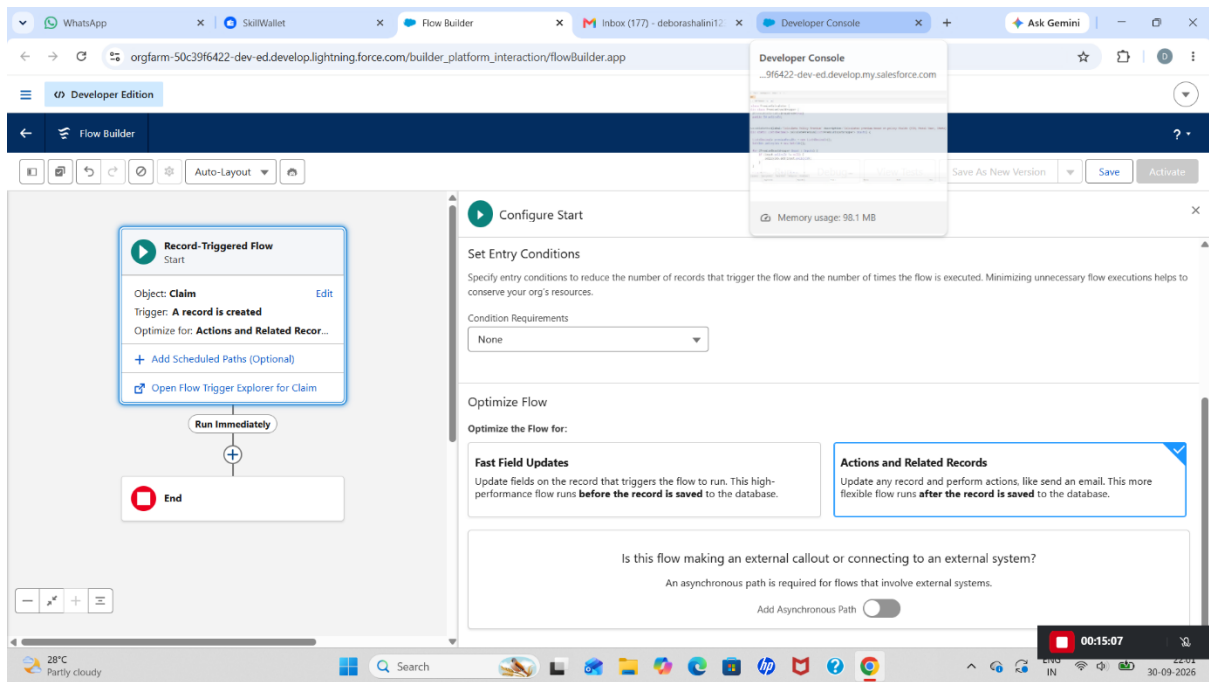
- Go to setup, in the quick find box search for flows and click on flows
- Find for AutoQuotingFlow and open in the flow builder
- Under the Create Records element,
- Add an action element after the create records element.
- Give the label as Calculate Premium Action
- API Name will be auto populated
- Under set Input values, For policy Id, Set the varPolicyId variable created in the previous step
- Under view output resources, check the manually assign variables checkbox and for the output placeholder, create a resource
- Resource type would be variable, API Name would be varCalculatedPremium, Data type would be currency, Decimal places are '2' and click Done.



- After the action element, add the update records element, Give the label as **Update Policy With Premium**
- For how to find records to update and set their values, Select Specify conditions to identify records, and set fields individually
- Select object as policy
- Select all conditions are met for filter criteria
- The conditions include:
- Record Id Equals varPolicyId variable
- Under set field values for policy record
- Assign varCalculatedPremium variable to the Premium fields
- Save the flow as a new version
- Activate the flow.

Create a Record-Triggered Flow (After Save) on the Claim object.

- Go to setup, In the quick find box, Search for flows and click on flows
- Click on New Flow, select Record Triggered Flow and Click on Create



- **Select Claim for an object**
- **Configure trigger when the record is created**
- **Optimize flow for action and Related Records**

- **Add Get Records element after the start and give the label as Get_Policy_RT**
- **Select policy for object**
- **Specify all conditions are met for conditional requirements**
- **The conditions include:**
- **Record Id Equals Triggering Claim → Policy**
- **Store only the first record**
- **Choose fields and assign variables for storing record data**
- **Select in Separate variables for where to store field values section**
- **Under select variables to store policy fields**
- **For recordTypeID field Create a variable, Resource type would be variable, API Name would be varPolicyRTName, Data type would be Text and click on Done**
- **Under Get Records Element, Add decision element**
- **Give the label as Route by Policy Type**
- **API name will be auto populated**
- **Select Decision Logic as Manually set conditions**
- **Specify the Outcomes:**
- **Outcome1:**

- **Label as Auto Claim Route**
- **API name will be auto populated**
- **Select All conditions for the conditional Requirements**
- **The condition includes:**
- **varPolicyRTName variable equals Auto**
- **Outcome2:**
- **Label as Property Claim Route**
- **API name will be auto populated**
- **Select All conditions for the conditional Requirements**
- **The condition includes:**
- **varPolicyRTName variable equals Auto**
- **Default Outcome:**
- **Label as Life Claim Route**
- **Under Auto Claim Route Decision path, add the update records element**
- **Label it as Assign to Auto Queue**
- **API name will be auto populated**
- **Under How to find records to update and set their fields, select specify conditions to identify records**
- **Select Claim as an object**
- **Select All conditions are true for Conditional requirements**
- **The conditions include:**
- **Record Id equals Triggering Claim → Record ID**
- **Under Set fields for the Claim Records**
- **Set the hardcoded auto queue 18 digit Id for Owner ID field**
- **Under Property Claim Route Decision path, add the update records element**
- **Label it as Assign to Property Queue**
- **API name will be auto populated**
- **Under How to find records to update and set their fields, select specify conditions to identify records**
- **Select Claim as an object**
- **Select All conditions are true for Conditional requirements**
- **The conditions include:**
- **Record Id equals Triggering Claim → Record ID**
- **Under Set fields for the Claim Records**
- **Set the hardcoded Property queue 18 digit Id for Owner ID field**
- **Under Life Claim Route Decision path, add the update records element**
- **Label it as Assign to Life Queue**
- **API name will be auto populated**
- **Under How to find records to update and set their fields, select specify conditions to identify records**
- **Select Claim as an object**
- **Select All conditions are true for Conditional requirements**
- **The conditions include:**
- **Record Id equals Triggering Claim → Record ID**

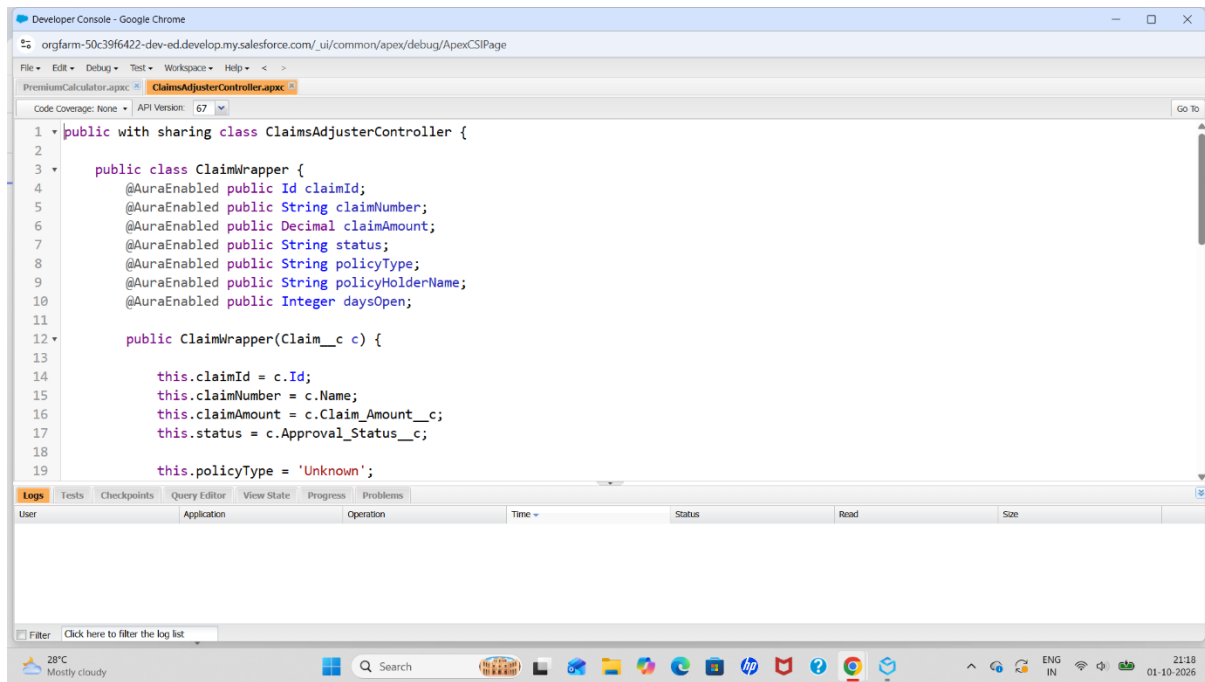
- Under Set fields for the Claim Records
- Set the hardcoded Life queue 18 digit Id for Owner ID field

Milestone 3: Claims Adjuster LWC Dashboard Development

Create a robust Apex class (ClaimsAdjusterController) with a method using SOQL to query all Claims owned by the logged-in user, joining related Policy and Customer data. The method must return a custom Wrapper Class listing key metrics.

- Click on setup icon, Right click on developer console, Open the link in new tab
- At the top left corner, click on the file, then click on new, and click on Apex class, Give the name as ClaimsAdjusterController and click on create.
- Paste the following code in the apex class:

```
public class ClaimsAdjusterController { public class ClaimWrapper { @AuraEnabled
public Id claimId; @AuraEnabled public String claimNumber; @AuraEnabled public
Decimal claimAmount; @AuraEnabled public String status; @AuraEnabled public
String policyType; @AuraEnabled public String policyHolderName; @AuraEnabled
public Integer daysOpen; public ClaimWrapper(Claim__c c, Policy__c p, Contact
customerRec) { this.claimId = c.Id; this.claimNumber = c.Name; this.claimAmount =
c.Claim_Amount__c; this.status = c.Approval_Status__c; Policy__c policyRecord =
(Policy__c)p; if (policyRecord != null && policyRecord.RecordType != null) {
this.policyType = policyRecord.RecordType.Name; } else { this.policyType =
'Unknown'; } this.policyHolderName = (customerRec != null) ? customerRec.Name : '';
this.daysOpen = (c.Date_of_Loss__c != null) ?
(Date.today().daysBetween(c.Date_of_Loss__c.date())) : 0; } }
@AuraEnabled(cacheable=true) public static List<ClaimWrapper>
getAssignedClaims() { List<ClaimWrapper> results = new List<ClaimWrapper>();
List<Claim__c> claims = [ SELECT Id, Name, Claim_Amount__c, Approval_Status__c,
Date_of_Loss__c, Policy__r.Name, Policy__r.RecordType.DeveloperName,
Policy__r.Customer__r.FirstName, Policy__r.Customer__r.LastName FROM Claim__c
WHERE OwnerId = :UserInfo.getUserId() ORDER BY Date_of_Loss__c DESC ]; for
(Claim__c claim : claims) { Policy__c policy = claim.Policy__r; Contact customer =
(policy != null && policy.Customer__r != null) ? (Contact)policy.Customer__r : new
Contact(); results.add(new ClaimWrapper(claim, policy, customer)); } return results; } }
```



- Save the file using CTRL + S
- This apex class will be handled in the LWC component

Create a component (claimsDashboardLwc) that uses the Apex method via the @wire service and implements client-side data filtering

- Open the new tab in chrome and search for lightning studio extension
- Open the first link and click on add to chrome, and confirm by clicking add Extension button
- Now open the Lightning studio extension from your salesforce org
- On the Top left corner, you will find Create New
- Click on Create New and Click on Lightning Web Component
- Enter the name of the Component as claimsDashboardLwc
- Check the isExposed checkbox
- Under target configurations, Select Lightning app page, Home page and Record page
- Click on Deploy
- Open the HTML file and paste the following code:

```

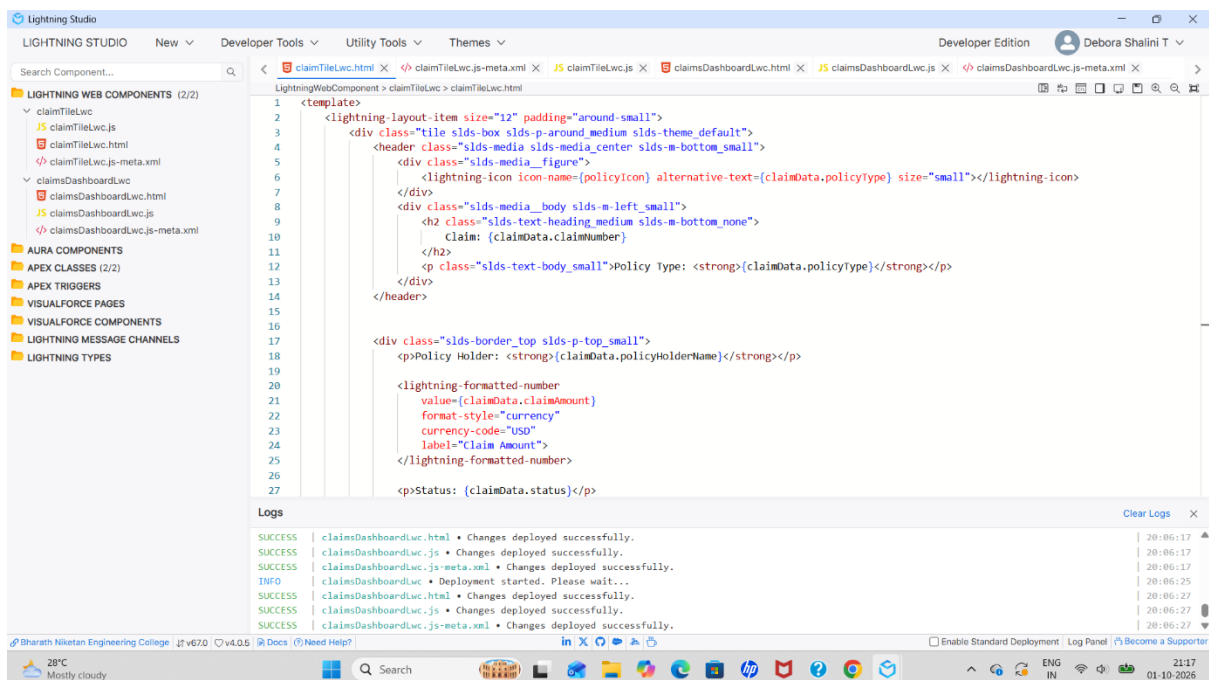
<template> <lightning-card title="My Assigned Claims Dashboard" icon-
name="standard:claim"> <div class="slds-m-around_medium">
<lightning-combobox name="policyFilter" label="Filter by Policy Type"

```

```

value="All" options={policyTypeOptions} onchange={handleFilterChange}
class="slds-m-bottom_medium" ></lightning-combobox> <template
if:true={visibleClaims}> <div class="slds-grid slds-wrap slds-grid_pull-
padded"> <template for:each={visibleClaims} for:item="claim"> <div
key={claim.claimId} class="slds-col slds-size_1-of-1 slds-medium-size_1-of-
2 slds-large-size_1-of-3 slds-p-horizontal_small"> <c-claim-tile-lwc claim-
data={claim}></c-claim-tile-lwc> </div> </template> </div> <p class="slds-
m-top_medium slds-text-heading_small">Total Claims Assigned:
{visibleClaims.length}</p> </template> <template if:false={visibleClaims}>
<p>No claims currently assigned to you.</p> </template> </div>
</lightning-card> </template>

```



➤ Open the JS file and paste the following code:

```

import { LightningElement, wire, track } from 'lwc'; // Import the Apex method import
getAssignedClaims from
'@salesforce/apex/ClaimsAdjusterController.getAssignedClaims'; export default class
ClaimsDashboardLwc extends LightningElement { // List to hold all data retrieved from
Apex @track allClaims = []; // List to display after filtering @track visibleClaims = [];
error; // Wire service to call the Apex method @wire(getAssignedClaims) wiredClaims({
error, data }) { if (data) { this.allClaims = data; this.visibleClaims = data; // Initially
display all claims this.error = undefined; } else if (error) { this.error = error;
this.allClaims = undefined; this.visibleClaims = undefined; console.error('Error
retrieving claims:', JSON.stringify(error)); } } // Client-side filtering logic (Required by
M3) handleFilterChange(event) { const selectedPolicyType = event.target.value; if

```

```
(selectedPolicyType === 'All') { this.visibleClaims = this.allClaims; } else { // Filter
claims where policyType matches the selected value this.visibleClaims =
this.allClaims.filter(claim => claim.policyType === selectedPolicyType ); } } get
policyTypeOptions() { // Options for the filter dropdown return [ { label: 'All Policy
Types', value: 'All' }, { label: 'Auto', value: 'Auto' }, { label: 'Property', value: 'Property'
}, { label: 'Life', value: 'Life' } ]; } }
```

- Open the meta-xml file and paste the following code:

```
<?xml version="1.0"?> <LightningComponentBundle
xmlns="http://soap.sforce.com/2006/04/metadata"> <apiVersion>63.0</apiVersion>
<isExposed>true</isExposed> <targets> <target>lightning__AppPage</target>
<target>lightning__HomePage</target> </targets> </LightningComponentBundle>
```

Create a reusable component (claimTileLwc) to display essential information for each claim

- Open the Lightning studio extension from your salesforce org
- On the Top left corner, you will find Create New
- Click on Create New and Click on Lightning Web Component
- Enter the name of the Component as claimsDashboardLwc
- Check the isExposed checkbox
- Under target configurations, Select Lightning app page, Home page and Record page
- Click on Deploy
- Open the HTML file and paste the following code:

```
<template> <lightning-layout-item size="12" padding="around-small"> <div
class="tile slds-box slds-p-around_medium slds-theme_default"> <header class="slds-
media slds-media_center slds-m-bottom_small"> <div class="slds-media__figure">
<lightning-icon icon-name={policyIcon} alternative-text={claimData.policyType}
size="small"></lightning-icon> </div> <div class="slds-media__body slds-m-
left_small"> <h2 class="slds-text-heading_medium slds-m-bottom_none"> Claim:
{claimData.claimNumber} </h2> <p class="slds-text-body_small">Policy Type:
<strong>{claimData.policyType}</strong></p> </div> </header> <div class="slds-
border_top slds-p-top_small"> <p>Policy Holder:
<strong>{claimData.policyHolderName}</strong></p> <lightning-formatted-number
value={claimData.claimAmount} format-style="currency" currency-code="USD"
```

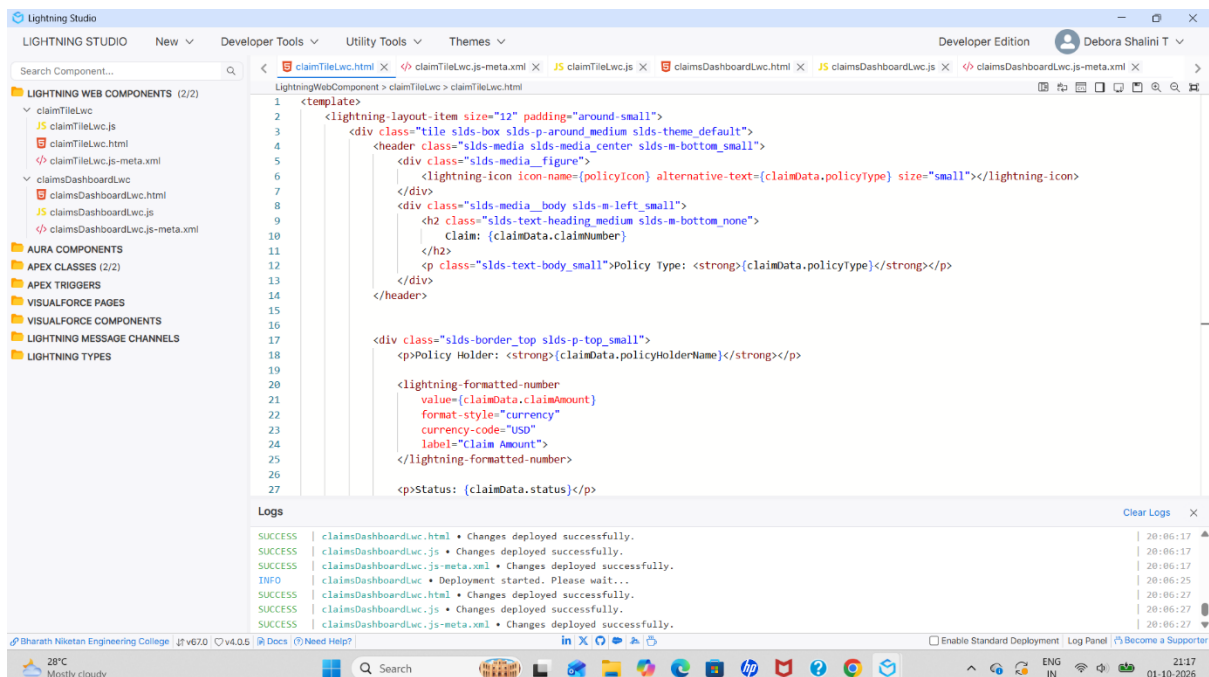
```
label="Claim Amount"> </lightning-formatted-number> <p>Status:
{claimData.status}</p> <p class="slds-text-body_regular"> Days Open: <strong
class="slds-text-color_error">{claimData.daysOpen}</strong> </p> </div> </div>
</lightning-layout-item> </template>
```

- Open the JS File and paste the following code

```
import { LightningElement, api } from 'lwc'; export default class ClaimTileLwc extends
LightningElement { // @api decorator exposes this property to the parent component
@api claimData; // Helper property to determine the correct icon based on Policy Type
get policyIcon() { if (this.claimData && this.claimData.policyType) { const type =
this.claimData.policyType.toLowerCase(); if (type === 'auto') return 'utility:truck'; if
(type === 'property') return 'utility:home'; if (type === 'life') return
'utility:connected_apps'; } return 'utility:help'; } }
```

- Open the meta-xml file and paste the following code

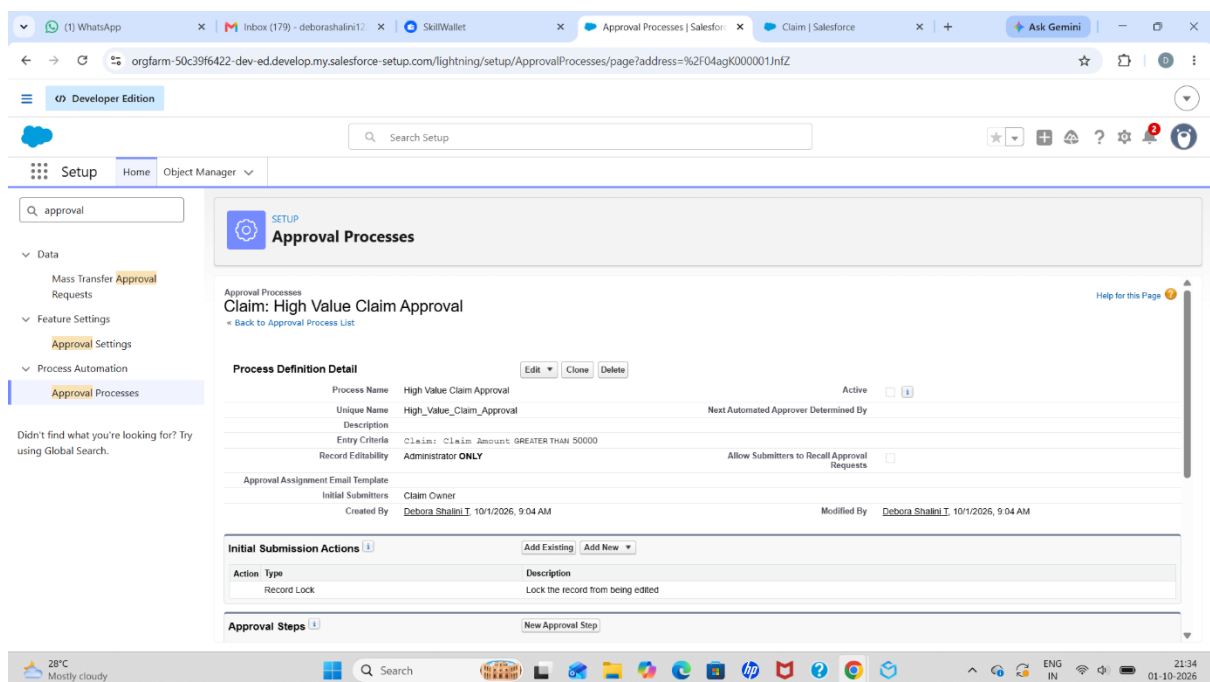
```
<?xml version="1.0"?> <LightningComponentBundle
xmlns="http://soap.sforce.com/2006/04/metadata"> <apiVersion>63.0</apiVersion>
<isExposed>true</isExposed> </LightningComponentBundle>
```



Milestone 4: Advanced Claim Processing, Security & Code Setup

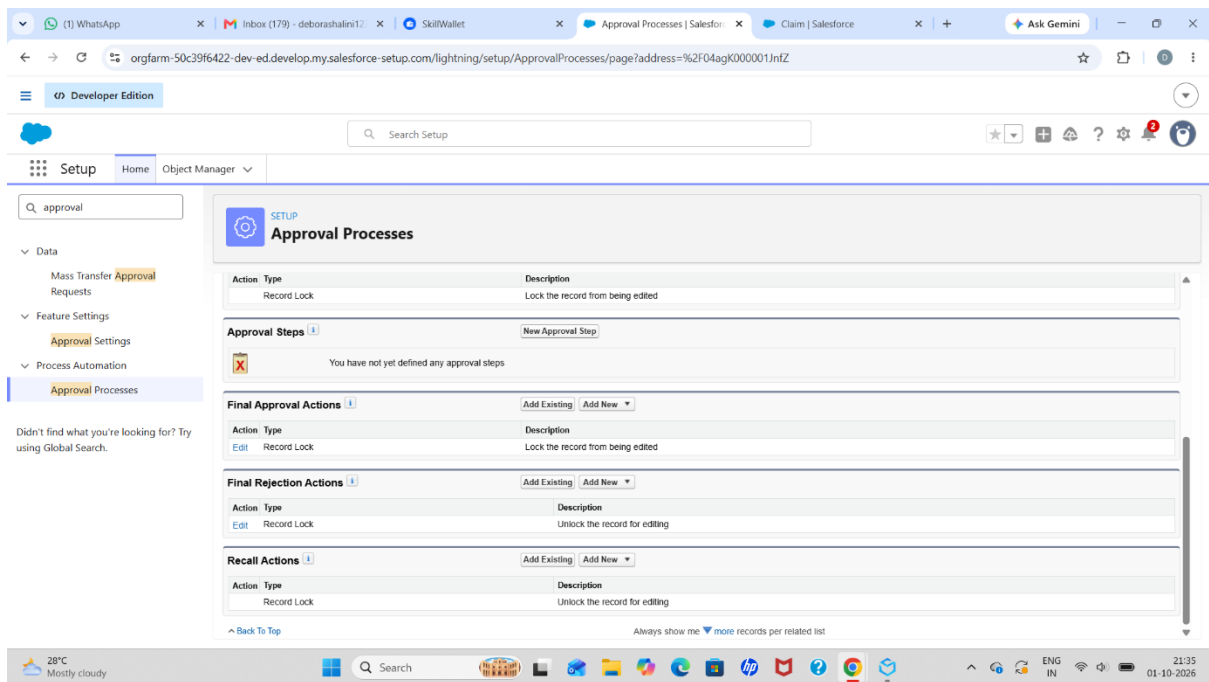
Modify the Claim object to include an Approval Status field.

- Go to set up, Click on object manager, search for claim object and click on claim object.
- In the left side panel, click on fields and relationships.
- Click on new
- Select Datatype as picklist
- Label it as Approval Status
- Select enter values radio button
- Enter New, Submitted for Approval, Approved and Rejected line by line
- Field Name will be auto populated
- Click on Next, Next and Save.



Configure the Approval Process to route first to the Senior Adjuster, then to the Department Manager, using two sequential steps.

- Go to setup, In the quick find box, Search for Approval process
- Select claim object for Manage approval process for
- Click on create new Approval Process and select use standard setup wizard
- Give the process name as High Value Claim Approval
- Unique Name will be auto populated
- Click on Next



- Under specify entry criteria, Select criteria are met for Use this approval process if
- Criteria includes:
- Claim amount greater than 50000
- Click on Next
- Under Record Editability properties, Select Administrators ONLY can edit the records during the approval process
- Click on Next
- Click on next again
- Under Approval Page Layout, Select Claim Number, Owner, Claim Amount, Adjuster, Policy fields to the selected fields from available fields
- Under security settings, Select Allow Approvers to access the page only from within the salesforce application.
- Click on Next

- Under Initial Submitters, Search for owner under submitter type, and select the claim owner as allowed submitters from available submitters
- Click on Save

Create Approval Steps

- **Step1:**
- Under initial submission actions, You can find Approval Steps
- Under approval steps, Click on New
- Give the Name as Step1 Senior Adjuster
- Unique name will be auto populated
- Click on Next
- Under specify step criteria, Select all records should enter the step
- Click on Next

The screenshot shows the Salesforce Setup interface. The left sidebar has a search bar with 'approval' entered. The 'Approval Processes' link is highlighted under the 'Process Automation' section. The main content area is titled 'New Approval Step' and shows 'Step 1. Enter Name and Description'. The form fields are as follows:

Enter Name and Description	
Approval Process Name	High Value Claim Approval
Name	Step1 Senior Adjuster
Unique Name	Step1_Senior_Adjuster
Description	
Step Number	1

Buttons for 'Next' and 'Cancel' are visible at the bottom right of the form.

- Under select Approver, Select Automatically assign to approver
- Under that select the Senior Adjuster User
- Under when multiple approvers are selected, select Approve or Reject based on the FIRST Response
- Click on Save
- Click on New again to create one more step
- Give the name as Step2 Manager Review
- Unique name will be auto populated
- Click on Next
- Under specify step criteria, Select all records should enter the step

➤ **Click on Next**

The screenshot shows the Salesforce Setup interface. The left sidebar contains a search bar with 'approval' and a list of categories: Data, Feature Settings, and Process Automation. Under 'Process Automation', 'Approval Processes' is selected. The main content area is titled 'Approval Processes' and shows 'Approval Step Edit: Senior Adjuster'. It is 'Step 1 of 3' with the title 'Step 1. Enter Name and Description'. A message says 'Enter a name, description, and step number for your new approval step.' Below this is a form with the following fields:

Enter Name and Description	
Approval Process Name	High Value Claim Approval
Name	Step1 Senior Adjuster
Unique Name	Step1_Senior_Adjuster
Description	

At the bottom right of the form are 'Save', 'Next', and 'Cancel' buttons. A red error message 'Required Information' is visible next to the 'Name' field.

- **Under select Approver, Select Automatically assign to approver**
- **Under that select the Claims Manager User**
- **Under when multiple approvers are selected, select Approve or Reject based on the FIRST Response**
- **Under Reject behaviour, Select Perform all rejection actions for this step AND all final rejection actions**
- **Click on Save**

Approval Processes | Salesforce

Developer Edition

Setup

approval

Data

Mass Transfer Approval Requests

Feature Settings

Approval Settings

Process Automation

Approval Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Step1 Senior Adjuster

Step 3. Select Assigned Approver

Specify the user who should approve records that enter this step. Optionally, choose whether the approver's delegate is also allowed to approve these requests.

Select Approver

☐ Let the submitter choose the approver manually.

☐ Automatically assign to queue. Senior Adjuster

☒ Automatically assign to approver(s).

User Senior Adjuster

Add Row Remove Row

When multiple approvers are selected:

☒ Approve or reject based on the FIRST response.

☐ Require UNANIMOUS approval from all selected approvers.

☐ The approver's delegate may also approve this request.

Previous Save Cancel

Approval Processes | Salesforce

Developer Edition

Setup

approval

Data

Mass Transfer Approval Requests

Feature Settings

Approval Settings

Process Automation

Approval Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Enter Name and Description

Enter a name, description, and step number for your new approval step.

Previous Approval Step Information

Step Number:	1
Name:	Step1 Senior Adjuster
Criteria:	
Assign To:	User - Senior Adjuster

Enter Name and Description

Approval Process Name: High Value Claim Approval

Name: Step2 Manager Review

Unique Name: Step2_Manager_Review

Description:

Step Number: 2

Next Cancel

Screenshot of the Salesforce Setup page for Field Updates. The browser tabs include WhatsApp, SkillWallet, Field Updates | Salesforce, and Claim | Salesforce. The URL is orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/WorkflowFieldUpdates/page/address=%2F04Y%2Fe%3Fpx2%3D04cgK000000qOuH%26TableEnumOrId%3D01lg... The page title is "Field Updates". The left sidebar shows "Setup" with "Home" and "Object Manager" tabs. The "Field Updates" section is active, showing "Update Status Submitted". The "Field Update Edit" form includes fields for Name, Unique Name, Description, Object, Field to Update, Field Data Type, and Re-evaluate Workflow Rules after Field Change. The "Specify New Field Value" section shows "Picklist Options" with a radio button for "The value above the current one".

Screenshot of the Salesforce Setup page for Field Updates. The browser tabs include WhatsApp, SkillWallet, Field Updates | Salesforce, and Claim | Salesforce. The URL is orgfarm-50c39f6422-dev-ed.develop.my.salesforce-setup.com/lightning/setup/WorkflowFieldUpdates/page/address=%2F04Y%2Fe%3Fpx2%3D04cgK000000qOuN%26TableEnumOrId%3D01lg... The page title is "Field Updates". The left sidebar shows "Setup" with "Home" and "Object Manager" tabs. The "Field Updates" section is active, showing "Update Status Rejected". The "Field Update Edit" form includes fields for Name, Unique Name, Description, Object, Field to Update, Field Data Type, and Re-evaluate Workflow Rules after Field Change. The "Specify New Field Value" section shows "Picklist Options" with a radio button for "The value above the current one".

- **Add video**
- **Under Initial Submission Actions, Click on Add New and select Field Update**
- **Give the name as Update Status Submitted**
- **Unique name will be auto populated**
- **Select object as Claim**
- **Select Approval Status for Field to Update**
- **Select Picklist as field datatype**
- **Under specify new field value, Select a Specific value and select submitted for Approval**
- **Click on Save**
- **Under Final Approval Actions, Click on Add New and select Field Update**
- **Give the name as Update Status Approved**
- **Unique name will be auto populated**
- **Select object as Claim**
- **Select Approval Status for Field to Update**
- **Select Picklist as field datatype**
- **Under specify new field value, Select a Specific value and select Approved**
- **Click on Save**

- **Under Final Rejection Actions, Click on Add New and select Field Update**
- **Give the name as Update Status Rejected**
- **Unique name will be auto populated**
- **Select object as Claim**
- **Select Approval Status for Field to Update**
- **Select Picklist as field datatype**
- **Under specify new field value, Select a Specific value and select Rejected**
- **Click on Save**
- **Activate the Approval Process.**

Create a Record-Triggered Flow (After Save) that checks if the Claim Amount is \$>\$50,000\$ and submits the record to an Approval Process

- **Go to setup, In the quick find box, search for flows and click on flows.**
- **Click on New Flow**
- **Select Record Triggered Flow**
- **Select object as Claim**
- **Configure Trigger When a Record is created or updated**
- **Select All conditions are true for conditional requirements**
- **The conditions include:**
- **Claim Amount Greater than 50000**

- **Approval Status Equals New**
- **Select Every time a record is updated and meets the conditional requirements for When to run the flow for Updated Records**
- **Optimize the flow for Action and Related Records**
- **Add an action element after the Start Element**
- **Label it as Submit Claim to Approval**
- **Under Set Input Values, For Record Id, Select Triggering Claim → Record ID**
- **For select Approval Process Name or Id, Enter the Approval Process Name(High Value Claim Approval)**

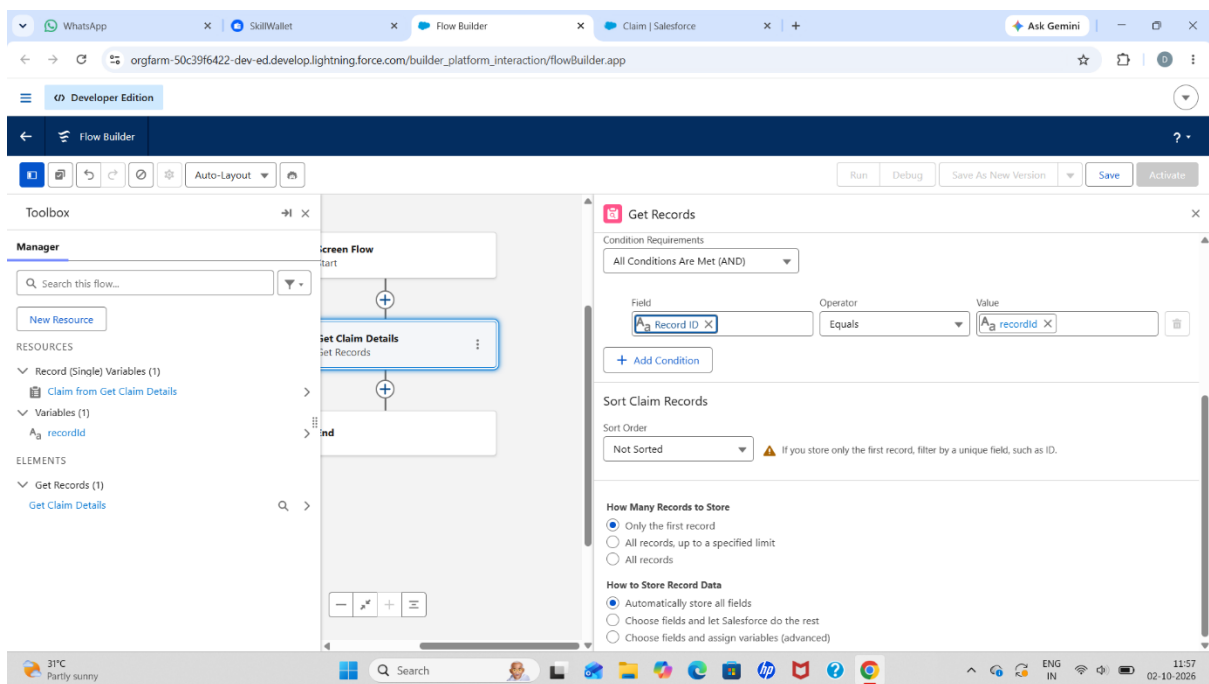
The screenshot shows the Salesforce Flow Builder interface in Developer Edition. The flow is titled 'Submission Automation Flow - V1'. The canvas displays a 'Record-Triggered Flow' starting with a 'Start' element, followed by a 'Run Immediately' connector, and then an action element labeled 'Submit Claim to Approval'. The right-hand pane shows the configuration for the 'Submit for Approval' action. The 'Label' is set to 'Submit Claim to Approval'. The 'API Name' is 'Submit_for_Approval_Action_1'. The 'Description' field is empty. Under 'Set Input Values', the 'Record ID' is set to 'Triggering Claim__c > Record ID' and the 'Approval Process Name Or ID' is set to 'High Value Claim Approval'. The flow is currently 'Inactive'.

This screenshot shows the same Salesforce Flow Builder interface, but the flow is now 'Active'. A green banner at the top of the canvas area states 'Your automation was activated.' The flow configuration on the right remains the same as in the previous screenshot. The status bar at the top right now shows 'Active' instead of 'Inactive'.

- **Save the Flow with Submission Automation Flow and Activate the flow**

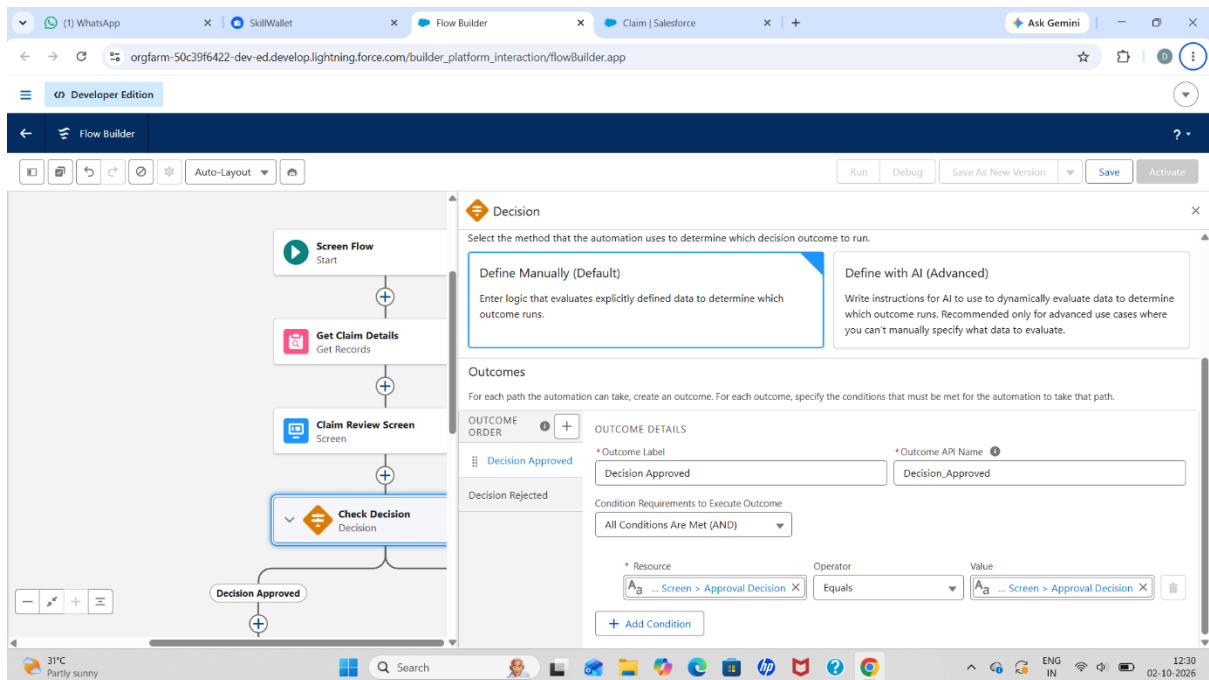
Implement a Screen Flow for the approvers to quickly review details and approve/reject directly from the Claim record

- **Go to setup, Search for flows in the quick find box and click on flows.**
- **Click on New Flow, Choose Screen Flow**
- **Create a New Resource by clicking on New Resource button in the toolbox manager**
- **Resource Type would be Variable**
- **API Name would be recordId**
- **Data type would be Text**
- **Under Availability outside the flow, Check Available for Input**
- **Click on Done**



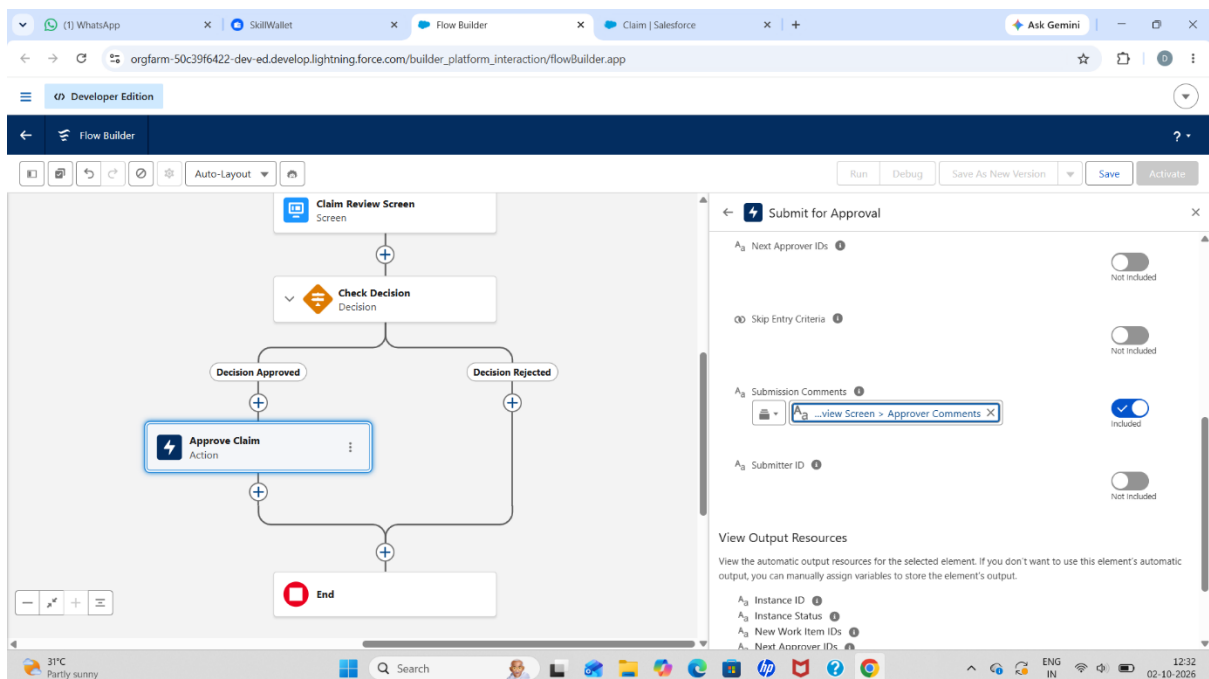
- **Add the Get Records Element below the screen**
- **Label it as Get Claim Details**
- **Select object as Claim**
- **Select All conditions are required for conditional Requirements**
- **The Condition include:**
- **RecordID equals variable recordId**
- **Store only the first record and**
- **Automatically store all field**

- Add a screen Element under the get Record element
- Label it as Claim Review Screen
- Add Display Text component to the flow canvas, and paste the following:
- "Claim ID: {!Get_Claim_Details.Name}"
- "Amount: {!Get_Claim_Details.Claim_Amount__c}"
- "Policy Type: {!Get_Claim_Details.Policy__r.Policy_Type__c}"
- Add the Radio buttons component, Label it as Approval Decision
- API Name would be auto populated
- For Choice, Create a resource, Resource type would be choice
- Give the choice label as Approve
- Select Datatype as Text
- Enter the choice value as Approve
- Enter the input label as Approve
- Click on Done
- Add one more Choice, Create a resource, Resource type would be choice
- Give the choice label as Reject
- Select Datatype as Text
- Enter the choice value as Reject
- Enter the input label as Reject
- Click on Done
- Add Long Text area into the flow canvas and Label it as Approver Comments
- API Name would be Auto Populated
- Click on Done



- Add Decision Element under the screen
- Label it as Check Decision

- **Select Manually Select Conditions**
 - **Set Outcome1:**
 - **Label it as Decision Approved**
 - **Select all conditions are met for conditional requirements**
 - **The condition includes:**
 - **Approve Decision From Claim Review Screen → Equals Approval Decision**
 - **Set default Outcome:**
 - **Label it as Decision Rejected**
-
- **Add Action Element Under Decision Approved Path**
 - **Search for the submit for approval action and choose it**
 - **Label it as Approve Claim**
 - **Under Set Input Values**
 - **Select variable recordID for Record ID**
 - **Include the submission comments and Enter Approver Comments from Claim Review Screen**
-
- **Add Action Element Under Decision Rejected Path**
 - **Search for the submit for approval action and choose it**
 - **Label it as Reject Claim**
 - **Under Set Input Values**
 - **Select variable recordID for Record ID**
 - **Include the submission comments and Enter Approver Comments from Claim Review Screen**



- Save and Activate the flow with Name Claim Approver Screen Flow

Create a Quick Action

- Go to setup, Click on Object Manager, Search for Claim Object and Select it
- In the left side panel, Click on Buttons, Links and Actions
- Click on Create New Action
- Select Action Type as Flow
- Select the flow that you have just created
- Label it as Approve/Reject Claim
- Name would be auto populated
- Click on Save
- Add the quick action to the claim page layout
- In the left panel, Click on page layouts and open the claim page layout
- Go to mobile & lightning actions, Find the Approve/Reject Claim and Drag it to the Salesforce Mobile and Lightning Experience Actions
- Click on Save

Write an extensive Apex Test Class for the ClaimsAdjusterController with 95%+ code coverage. Include test methods for complex claim filtering and data retrieval logic

- Click on the setup icon, Right click on Developer Console and click on open link in new tab.
- On the top left corner, click on file, click on new, click on Apex class and give it a name ClaimsAdjusterControllerTest
- Paste the following code:

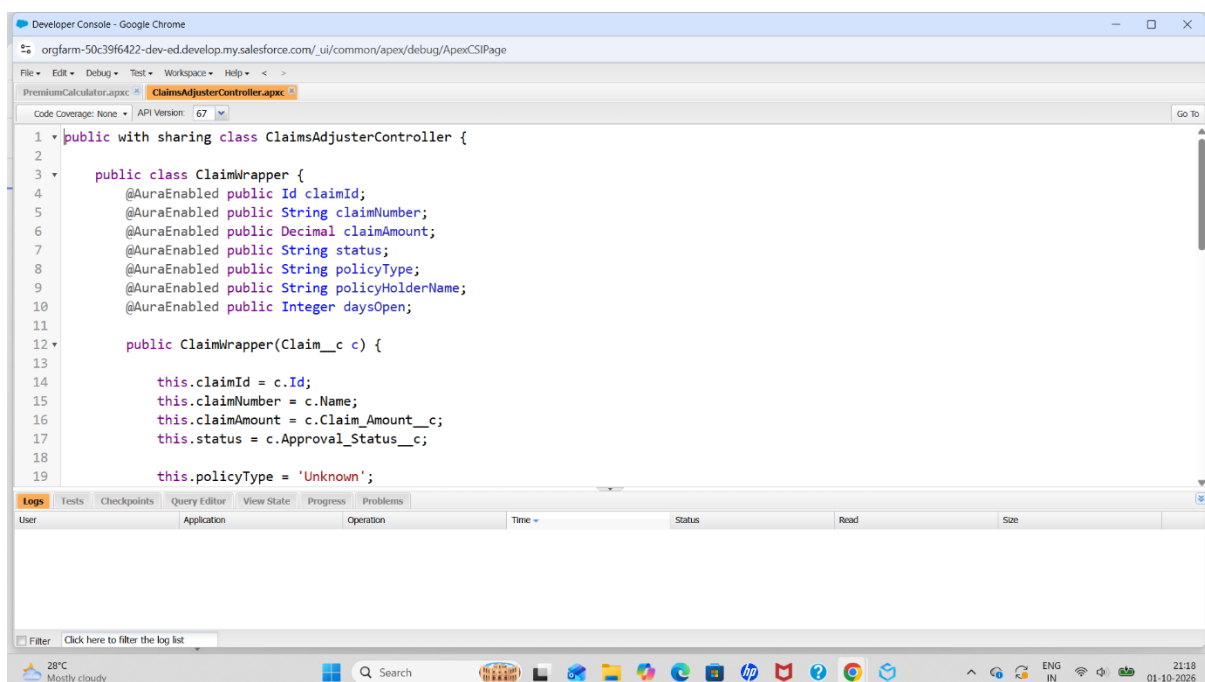
```
@isTest private class ClaimsAdjusterControllerTest { @testSetup static void
setupTestData() { // Create a real active test user Profile p = [SELECT Id FROM Profile
WHERE Name = 'Standard User' LIMIT 1]; User u = new User( FirstName = 'Test',
LastName = 'User', Alias = 'tuser', Username = 'testuser1234343@example.com.test',
Email = 'testuser123@example.com', TimeZoneSidKey = 'America/Los_Angeles',
LocaleSidKey = 'en_US', EmailEncodingKey = 'UTF-8', LanguageLocaleKey = 'en_US',
ProfileId = p.Id ); insert u; // Query Policy__c RecordType Id policyRT =
Schema.SObjectType.Policy__c.getRecordTypeInfoByDeveloperName().get('Auto')
.getRecordTypeId(); Id claimRT =
Schema.SObjectType.Claim__c.getRecordTypeInfoByDeveloperName()
.get('Accident').getRecordTypeId(); // Create Contact owned by same user Contact con
```

```

= new Contact( FirstName = 'John', LastName = 'Doe', OwnerId = u.Id ); insert con; //
Create Policy owned by same user Policy__c policy = new Policy__c( Customer__c =
con.Id, RecordTypeId = policyRT, OwnerId = u.Id, VIN__c = '1A903284375893532' );
insert policy; // Create Claim owned by same user Claim__c claim = new Claim__c(
Policy__c = policy.Id, RecordTypeId = claimRT, Claim_Amount__c = 5000,
Date_of_Loss__c = Date.today().addDays(-10), OwnerId = u.Id ); insert claim;
System.debug('Claim inserted: '+claim.Id); } @isTest static void
testGetAssignedClaims() { User u = [SELECT Id FROM User WHERE Alias = 'tuser'
LIMIT 1]; System.runAs(u) { Test.startTest();
List<ClaimsAdjusterController.ClaimWrapper> results =
ClaimsAdjusterController.getAssignedClaims(); System.debug('Result is: '+results);
Test.stopTest(); //System.assert(results.size() > 0, //'Expected at least one wrapper
returned'); ClaimsAdjusterController.ClaimWrapper wrap = results[0];
System.assertNotEquals(null, wrap.claimId); System.assertNotEquals(null,
wrap.claimNumber); System.assertEquals('New', wrap.status);
System.assertEquals(5000, wrap.claimAmount);
System.assert(wrap.policyHolderName.contains('John'));
System.assertNotEquals('Unknown', wrap.policyType); } } }

```

- Save the file using CTRL+S
- On the top right corner, you will find Run Test, Click on it to run the test class
- To check the test results, Down go for tests and you can see the how much percentage it covers your apex class



Refine the Sharing Rules to ensure Adjusters only see claims in their assigned territory/state

- Go to setup, Search for flows in the quick find box and click on flows
- Click on New Flow
- Select Record Triggered Flow
- Select the object as claim
- Configure trigger when a record is created or updated
- Set entry conditions as none
- Optimize the flow for Updates
- Add get Records element into the flow canvas
- Label it as get_policy_records
- API name will be auto populated
- Set filter as None
- Store all the records
- Select Choose fields and let salesforce do the rest for How to store Record Data
- Under Select Policy fields to store fields in Variable, Select Policy State Field

The screenshot shows the Salesforce Setup interface for 'Sharing Settings'. The left sidebar includes 'Setup', 'Home', and 'Object Manager'. The main content area is titled 'Setup Claim Sharing Rule'. It includes a search bar, a 'Help for this Page' link, and a 'Note' about roles and subordinates. The configuration is divided into three steps:

- Step 1: Rule Name**
 - Label: Share Records based on P
 - Rule Name: Share_Records_based_on_1
 - Description: (empty)
- Step 2: Select your rule type**
 - Rule Type: Based on criteria (selected)
- Step 3: Select which records to be shared**

Criteria	Field	Operator	Value
	Policy Account Holder State	equals	CA
	--None--	--None--	
	--None--	--None--	

The bottom of the screen shows a Windows taskbar with various application icons and a system tray with a rain warning, search bar, and date/time (14:01, 02-10-2026).

- Add the Loop Element into the canvas after the get Records element
- Label it as Claim_Filed_update
- API name will be auto populated
- Under Select Collection Variables, Select policies from Get Policies from Get Policy Records
- Under Specify direction for Iterating over collection, Select first item to last item

- Under For each, Add the update records element into the canvas
- Label it as Claim_Filed *state*update
- API name will be auto populated
- Set filter conditions to NONE
- Under Set field values for the claim record, Select policy state from current loop and set it to the Policy Account Holder State field

The screenshot shows the Salesforce Flow Builder interface. The flow is named "Claim Policy Holder State Update - V1". The flowchart consists of the following steps:

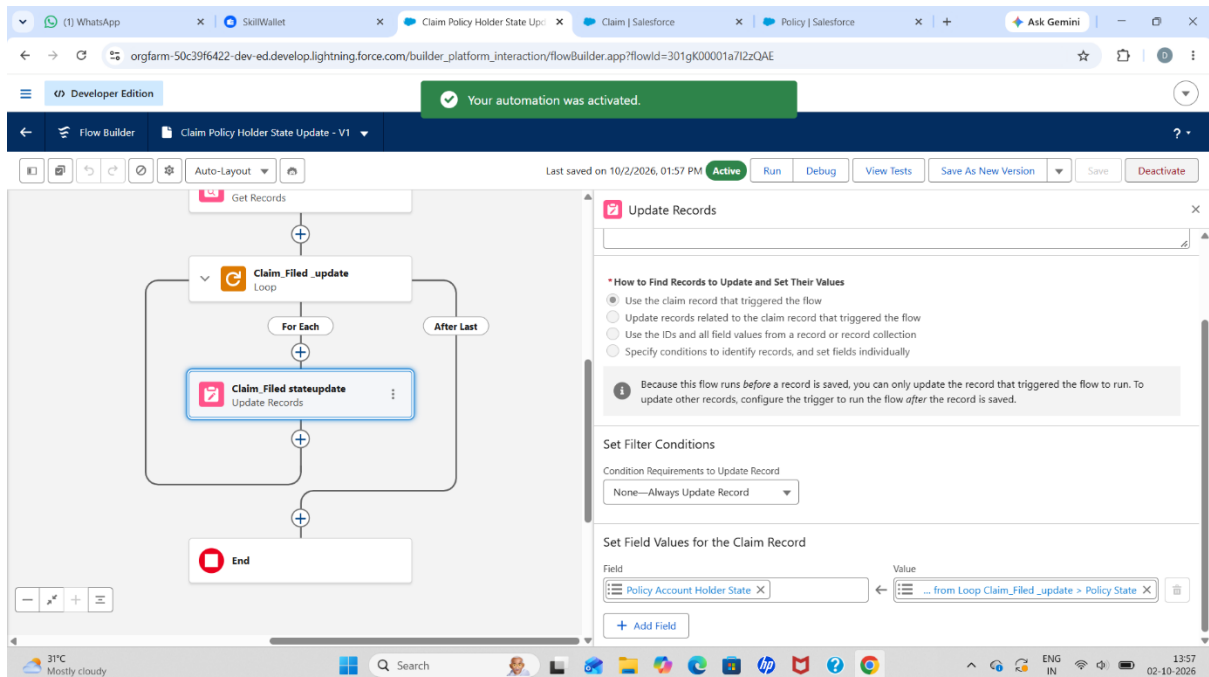
- get_policy_records** (Get Records)
- Claim_Filed_update** (Loop)
 - For Each**
 - Update Records 1** (Update Records)
 - After Last**
- End**

The right sidebar displays the configuration for the "Loop" element:

- * Label:** Claim_Filed_update
- * API Name:** Claim_Filed_update
- Description:** (Empty)
- Select Collection Variable:** Policies from get_policy_records
- Specify Direction for Iterating Over Collection:**
 - * Direction:**
 - ☒ First item to last item
 - ☐ Last item to first item

The screenshot shows the Salesforce Flow Builder interface with the "get_policy_records" action selected. The flowchart is the same as the previous screenshot. The right sidebar displays the configuration for the "Get Records" element:

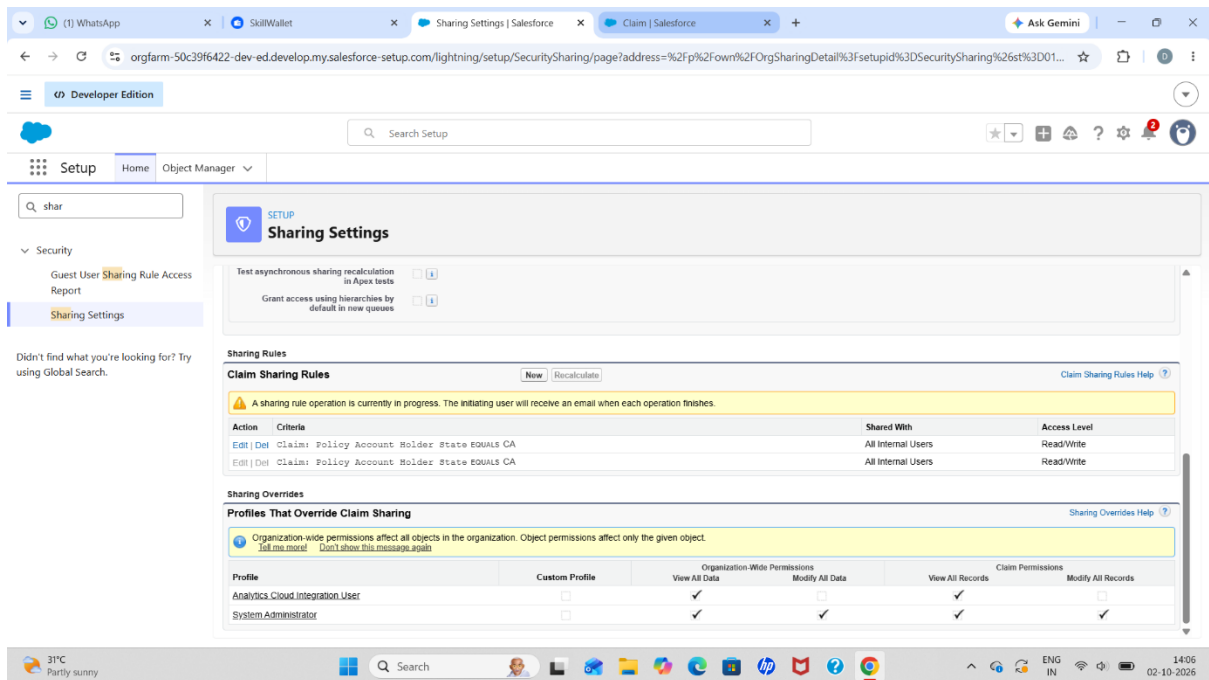
- * Label:** get_policy_records
- * API Name:** get_policy_records
- Description:** (Empty)
- Get Records of This Object:**
 - * Object:** Policy
- Filter Policy Records:**
 - Condition Requirements:** None—Get All Policy Records
- Sort Policy Records:**
 - Sort Order:** Not Sorted



- **Save and Activate the flow with name Claim Policy Holder State Update**

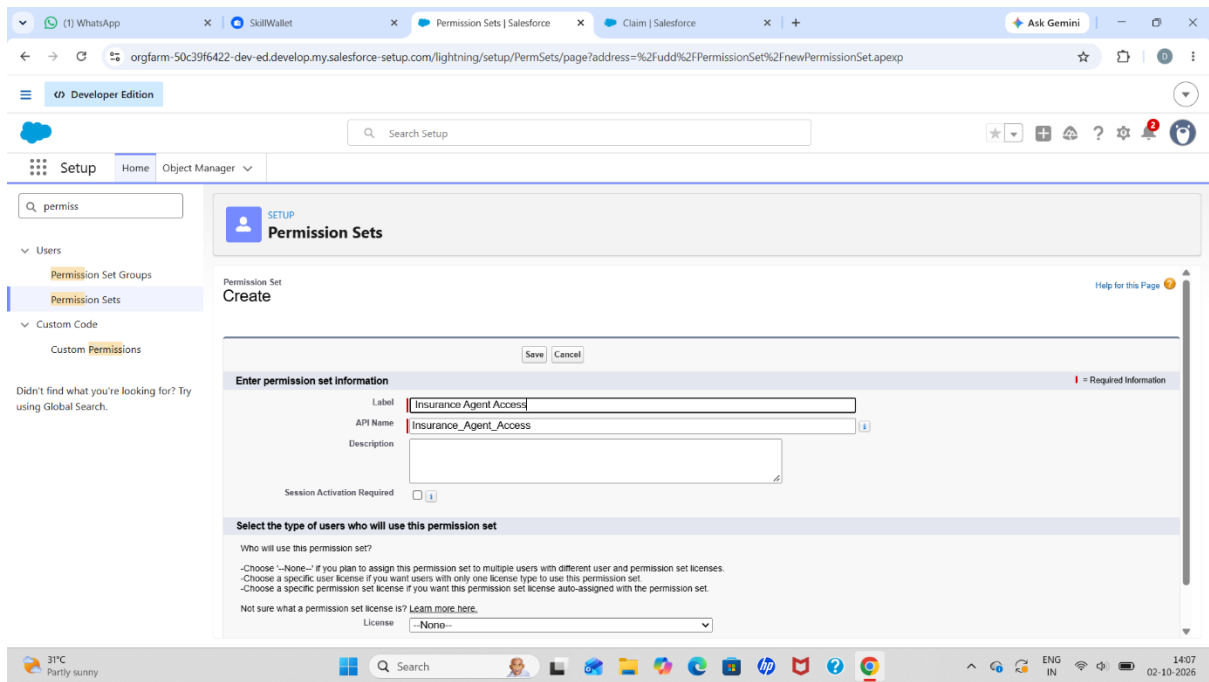
Define Sharing Rules For Claim Object

- **Go to Set up, In the quick find box, Search for Sharing Settings**
- **Click on Sharing settings**
- **Under Manage Sharing Settings for, Select Claim Object**
- **Scroll down to find Claim Sharing Rules**
- **Click on new**
- **Label it as Share Records based on Policy State CA**
- **Rule Name will be Auto Populated**
- **Set the criteria to Policy Account Holder State Equals CA**
- **Give Access Level as Read/Write**
- **Click on Save**
- **Again Click on New**
- **Label it as Share Records based on Policy State**
- **Rule Name will be Auto Populated**
- **Set the criteria to Policy Account Holder State Equals TX**
- **Give Access Level as Read/Write**
- **Click on Save**



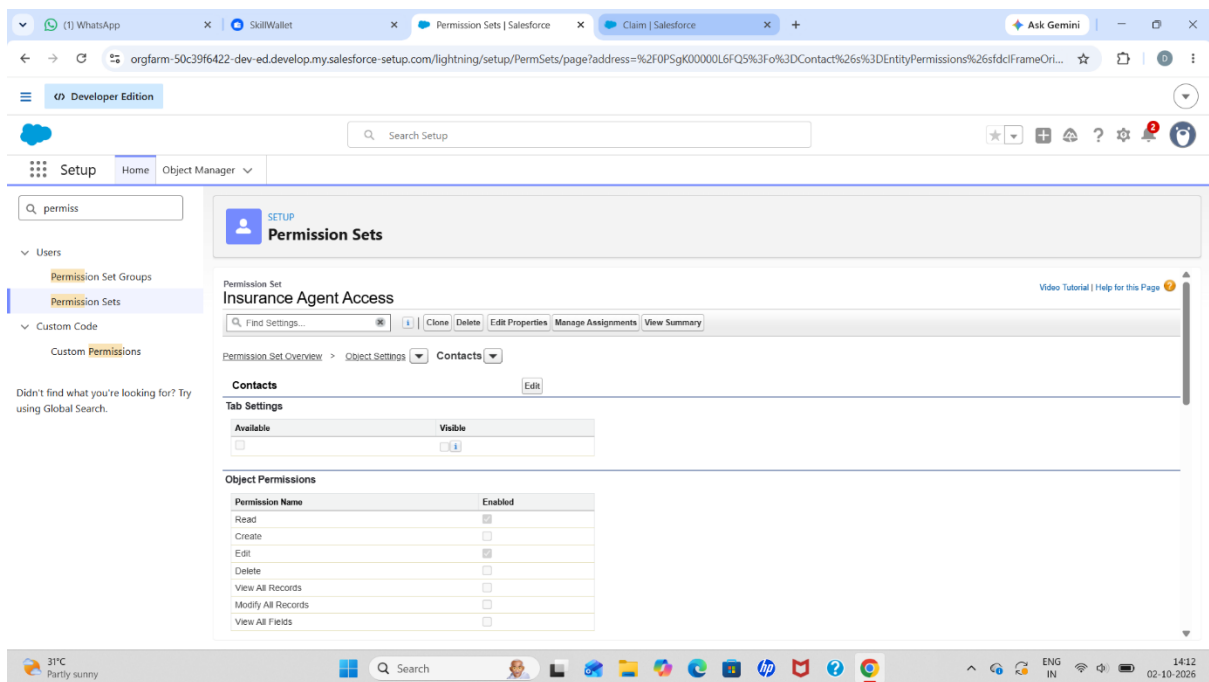
Finalize all Permission Sets to differentiate access between Agents (Quoting only), Adjusters (Claim handling), and Managers (Reporting/Approvals)

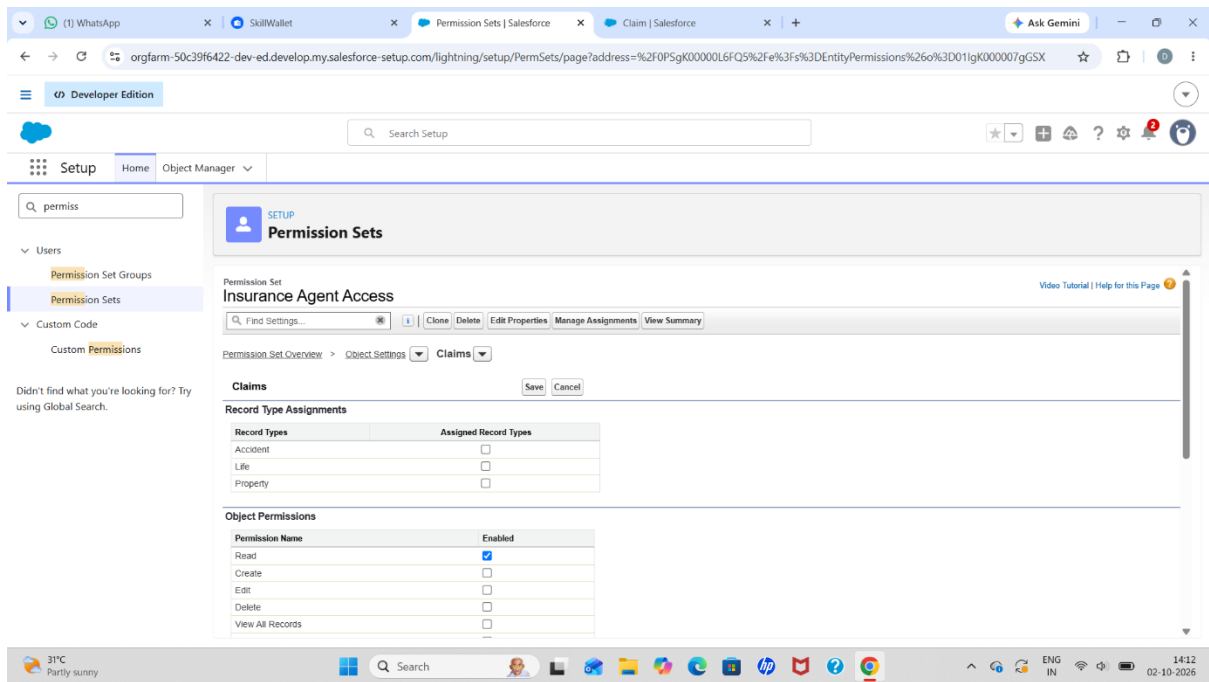
- Go to setup, In the quick find box, Search for Permission Sets, Click on Permission Sets
- Click on New
- Give the Label as Insurance Agent Access
- API Name will be Auto Populated
- Give the access for Objects
- Give Create, Read for Policy object
- Give Read, Edit to contact Object
- Give Read only to Claim object
- Make Flow User Enabled under System permissions
- Make PremiumCalculator Apex class enabled



Create Claims Manager Access Permission Sets

- Click on New
- Give the Label as Claims Manager Access
- API Name will be Auto Populated





- Give the access to objects
- Give Read Only for Policy object
- Give Read, Edit to contact Object
- Give Read, Edit, Delete to Claim object
- Make Run Reports Enabled under System permissions
- Make Manage Approvals enabled under system permissions

Create Claims Adjuster Access Permission Sets

- Click on New
- Give the Label as Claims Adjuster Access
- API Name will be Auto Populated
- Give the access to objects
- Give Read Only for Policy object
- Give Read, Edit to Claim object
- Make ClaimsAdjusterController Enabled under Apex Classes

Screenshot of the Salesforce Setup page for the "Claims Adjuster Access" permission set. The page shows the "Apex Class Access" section with a table listing the Apex Class Name "ClaimsAdjusterController" and the Installed Package.

Permission Set Overview > Apex Class Access

Items per page: 50 Items

View: All

Apex Class Name	Installed Package
ClaimsAdjusterController	

Screenshot of the Salesforce Setup page for the "Claims Manager Access" permission set. The page shows the "Permission Set Overview" section with details about the API Name, Namespace Prefix, Created By, and Last Modified By. Below this, there are sections for "Assigned Apps", "Assigned Connected Apps", "Object Settings", "App Permissions", and "Apex Class Access".

Permission Set Overview

Description	API Name	Claims_Manager_Access
License	Namespace Prefix	
Session Activation Required	Created By	Deborah Shalini.T. 10/2/2026, 2:04 AM
Permission Set Groups Added To	Last Modified By	Deborah Shalini.T. 10/2/2026, 2:16 AM

Apps

Assigned Apps
Settings that specify which apps are visible in the app menu

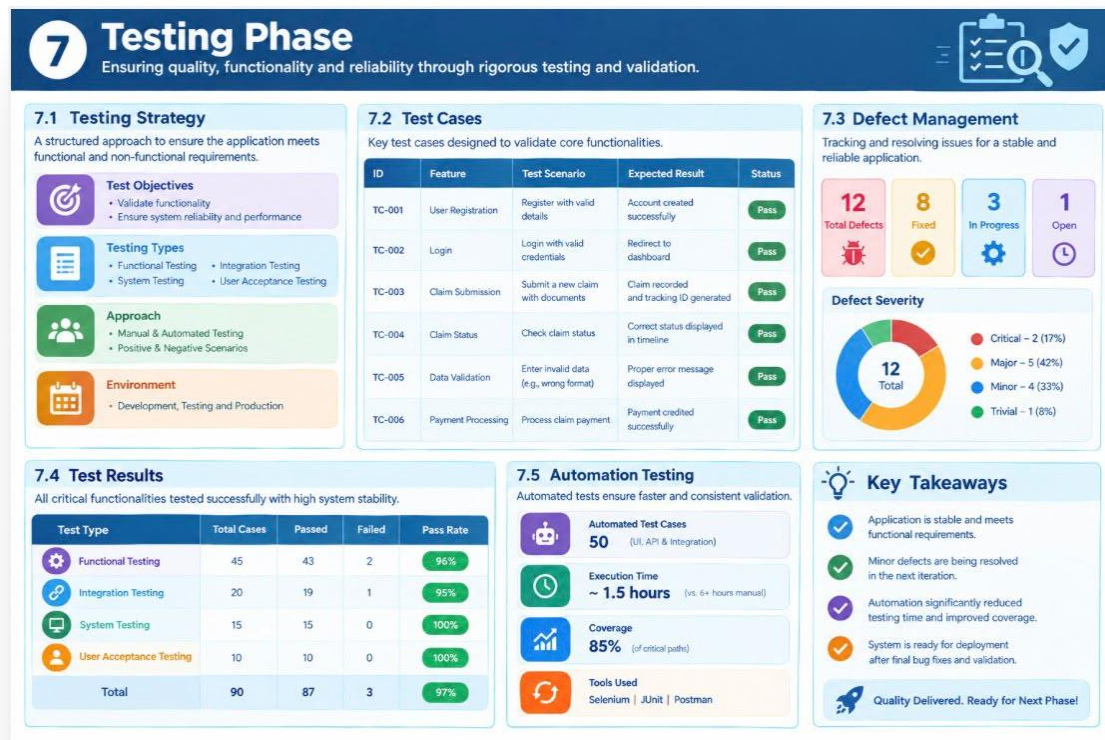
Assigned Connected Apps
Settings that specify which connected apps are visible in the app menu

Object Settings
Permissions to access objects and fields, and settings such as tab availability

App Permissions
Permissions to perform app-specific actions, such as "Manage Call Centers"

Apex Class Access
Permissions to execute Apex classes

7. FUNCTIONAL AND PERFORMANCE TESTING



7.1 Functional Testing

Functional testing verifies whether the system performs the required business functions correctly.

Test ID	Test Case	Test Action	Expected Result	Result
TC01	Create Policy	Create a valid policy record	Policy should be created successfully	Pass
TC02	Create Policy Holder	Enter valid policy holder information	Policy holder information should be saved	Pass
TC03	Create Claim	Create a claim against a valid policy	Claim should be created successfully	Pass
TC04	Submit Claim	Submit a valid claim	Claim should enter the appropriate processing stage	Pass
TC05	Claim Routing	Trigger configured routing conditions	Claim should be routed according to configured rules	Pass
TC06	Claim Assignment	Process a routed claim	Claim should be assigned to the appropriate queue/user	Pass
TC07	Open Dashboard	Open the claims dashboard	Dashboard should load and display claim information	Pass
TC08	Filter Claims	Apply a claim filter	Only matching claims should be displayed	Pass
TC09	Update Claim	Modify claim information	Claim information should be updated successfully	Pass
TC10	Security Access	Attempt access with an unauthorized user	Restricted information should not be accessible	Pass
TC11	Apex Processing	Execute the relevant Apex process	Required business logic should execute successfully	Pass

7.2 Performance Testing

Performance testing evaluates how efficiently the system operates.

Important areas include:

- Dashboard loading time.
- Claim retrieval.
- Apex execution.
- Flow execution.
- Filtering performance.

- **Large-volume data processing.**
- **User response time.**

The system should avoid unnecessary queries and processing and should follow Salesforce development best practices.

8. ADVANTAGES AND DISADVANTAGES

Advantages

1. Centralized Data

Policy, customer, and claim information can be maintained in a centralized platform.

2. Automation

Automated routing and processing reduce repetitive manual activities.

3. Improved Visibility

The claims dashboard provides adjusters with a consolidated view of claims.

4. Faster Claim Processing

Automation can reduce unnecessary manual steps.

5. Better Data Management

Structured Salesforce records help maintain consistent information.

6. Security

Salesforce security features can be used to control access to sensitive information.

7. Scalability

The solution can be extended with additional insurance products and business processes.

8. Better User Experience

The LWC dashboard provides an interactive interface for claims adjusters.

Disadvantages

1. Salesforce Dependency

The solution depends on the Salesforce platform.

2. Configuration Complexity

Complex insurance business rules can require significant configuration.

3. Development Skills

Apex, LWC, Flow, and Salesforce administration knowledge may be required.

4. Platform Limits

Salesforce has platform limits that must be considered during development.

5. Initial Development Effort

Building the complete data model, automation, dashboard, and security model requires considerable development effort.

9. CONCLUSION

The Multi-Line Insurance Policy and Claims Management System provides a centralized Salesforce-based approach for managing insurance policies and claims.

The project combines a structured data model, automated claim routing, an LWC claims dashboard, advanced claim processing, Apex development, and security controls. The four milestones provide an end-to-end development process from data configuration to advanced claim processing and testing.

Overall, the project demonstrates how Salesforce technologies can be used to design, automate, and manage a modern insurance policy and claims workflow.

10. FUTURE SCOPE

The project can be further enhanced with the following features:

1. AI-Based Claim Assessment

AI can be introduced to analyze claim information and assist adjusters in identifying relevant patterns.

2. Automated Document Processing

OCR and document-processing capabilities can be added to extract information from submitted documents.

3. Customer Self-Service Portal

A customer portal could allow policy holders to:

- **View policies.**
- **Submit claims.**
- **Upload documents.**
- **Track claim status.**

4. Mobile Application

A mobile interface could allow adjusters and customers to access relevant information from mobile devices.

5. Advanced Analytics

Additional dashboards could provide:

- **Claim trends.**
- **Processing times.**
- **Policy statistics.**
- **Claim volumes.**

6. Notification System

Email or in-app notifications could inform users when:

- **A claim is submitted.**
- **A claim is assigned.**
- **Additional information is required.**
- **A claim status changes.**

7. Integration

The system could integrate with:

- **Payment systems.**
- **External insurance databases.**
- **Document management systems.**
- **Customer communication platforms.**

8. Predictive Analytics

Historical claim data could be analyzed to identify patterns and support operational decision-making.

11. APPENDIX

Appendix A – Major Project Components

Data Components

- **Policy**
- **Policy Holder**
- **Claim**
- **Claim Status**
- **Claim Assignment**
- **Insurance Type**

Automation Components

- **Salesforce Flow**
- **Claim routing**
- **Field updates**
- **Assignment logic**
- **Validation**

Development Components

- **Apex Classes**
- **SOQL Queries**

Lightning Web Components

- **JavaScript**
- **HTML**
- **CSS**

Security Components

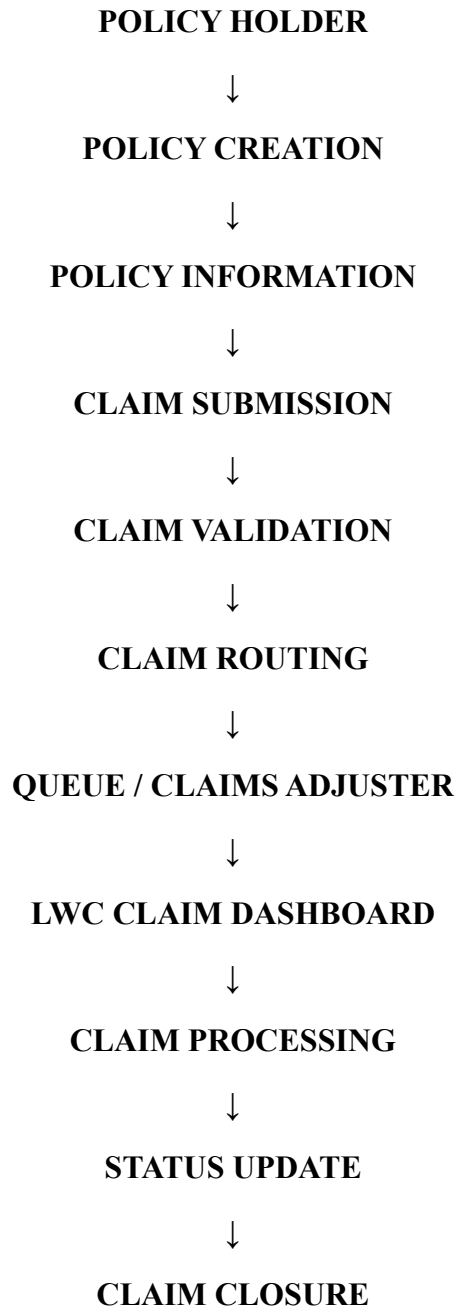
- **Profiles**
- **Permission Sets**
- **Object Permissions**
- **Field-Level Security**
- **Record-Level Access**

Appendix B – Milestone Summary

Milestone	Focus	Main Outcome
M1	Core Data Model & Policy Configuration	Structured insurance and policy data
M2	Complex Policy Insurance & Claim Routing Automation	Automated claim processing and routing
M3	Claims Adjuster LWC Dashboard Development	Interactive claims dashboard
M4	Advanced Claim Processing, Security & Code Setup	Advanced processing, Apex and secure system

Appendix C – Overall System Flow

Step	System Stage	Description
1	Policy Holder	Provides or maintains insurance information
2	Policy	Policy record is created and maintained
3	Claim	Claim is submitted against the relevant policy
4	Validation	Claim information is checked
5	Routing	Claim is routed based on configured business rules
6	Adjuster / Queue	Claim is assigned for review
7	LWC Dashboard	Adjuster views and filters claim information
8	Claim Processing	Claim is processed and updated
9	Status Update	Claim status is recorded
10	Claim Closure	Claim reaches its final processing stage



Appendix D – Project Technology Summary

- **Platform: Salesforce**
- **Automation: Salesforce Flow**
- **Backend: Apex**
- **Database/Query: Salesforce Objects + SOQL**
- **Frontend: Lightning Web Components**
- **Frontend Technologies: HTML, CSS, JavaScript**
- **Security: Salesforce Security Model**
- **Development: Salesforce Developer Tools / VS Code**